LAPORAN PRAKTIKUM TEKNOLOGI IOT II

TUGAS MODUL 7

Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

Disusun Oleh:

Nama : Naela Deswita

NPM : 25782106

Kelas : 3D-TRI

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

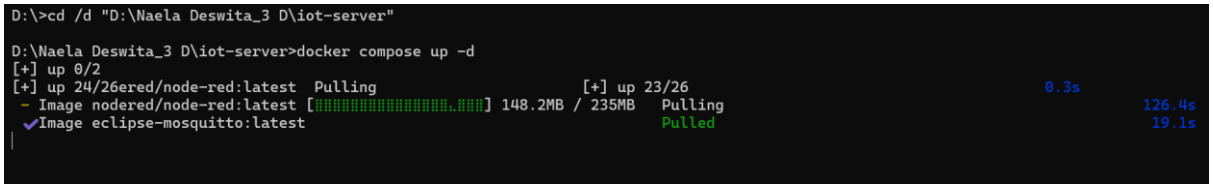
POLITEKNIK NEGERI LAMPUNG

2026/2027

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

1. Proses menjalankan Docker Compose

Saya memulai praktikum dengan menjalankan perintah docker compose up -d melalui CMD pada folder proyek iot-server. Perintah ini digunakan untuk menjalankan layanan yang dibutuhkan dalam membangun server IoT lokal. Docker kemudian menyiapkan container Mosquitto sebagai broker MQTT dan Node-RED sebagai platform untuk mengolah data serta membuat aturan otomasi.



2. Container Mosquitto dan Node-RED berhasil dijalankan

Setelah proses Docker Compose selesai, container mqtt-broker dan node-red berhasil dibuat dan dijalankan. Kedua layanan ini menjadi bagian utama dalam praktikum, yaitu Mosquitto untuk menerima dan meneruskan pesan MQTT, sedangkan Node-RED digunakan untuk mengolah data sensor dan membuat sistem kendali LED.



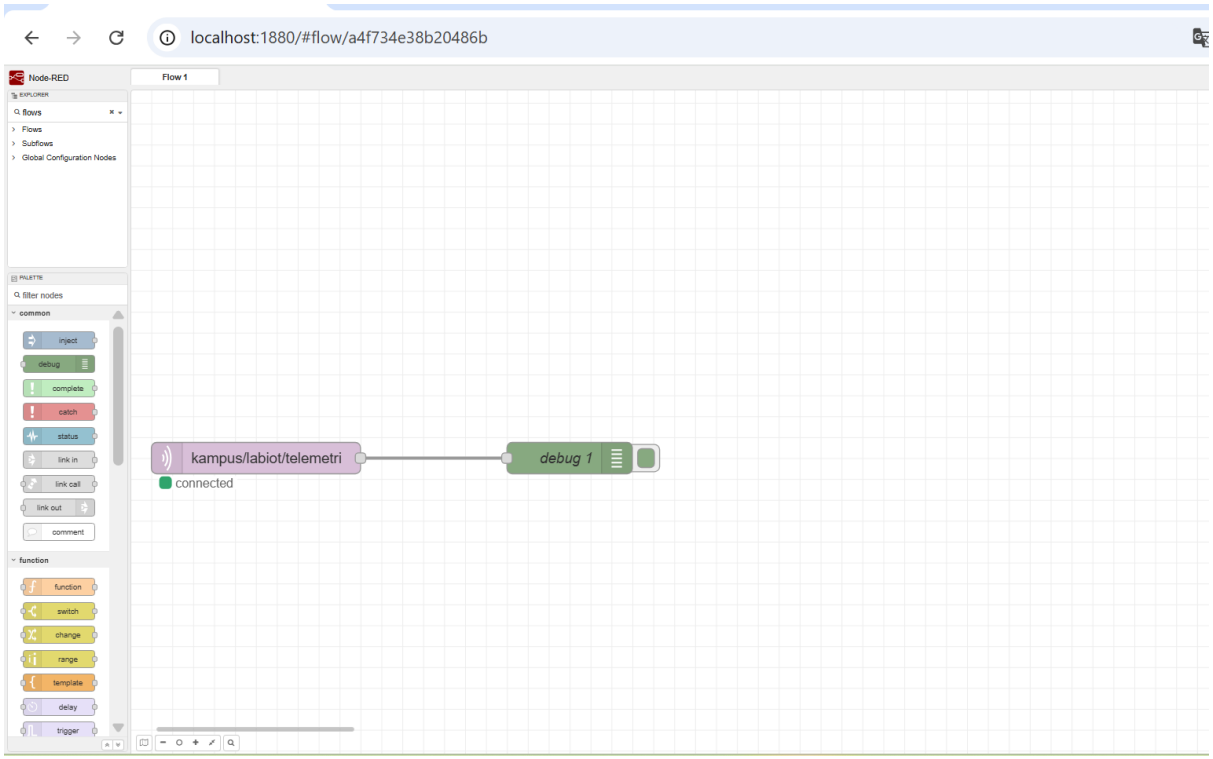
3. Memeriksa container menggunakan perintah docker ps

Saya menjalankan perintah docker ps untuk memeriksa container yang sedang aktif. Dari hasil pemeriksaan, container Mosquitto dan Node-RED terlihat berjalan dengan port yang telah ditentukan. Ada juga container dari praktikum sebelumnya, tetapi layanan yang digunakan dalam Modul 7 ini adalah Mosquitto dan Node-RED.



4. Membuka antarmuka Node-RED melalui port 1880

Selanjutnya, saya membuka Node-RED melalui browser dengan alamat http://localhost:1880. Halaman Node-RED berhasil dibuka dan menampilkan kanvas untuk menyusun alur program. Pada tahap ini, saya mulai menyiapkan alur yang nantinya digunakan untuk menerima data dari broker MQTT dan menampilkannya melalui node Debug.



5. Tampilan Docker Desktop dan log broker

Saya juga memeriksa layanan melalui Docker Desktop untuk memastikan container yang digunakan dalam praktikum sudah berjalan. Log broker digunakan untuk melihat aktivitas koneksi dan pesan yang berkaitan dengan layanan MQTT. Pemeriksaan ini membantu saya mengetahui kondisi broker ketika NodeMCU dan Node-RED mulai berkomunikasi.



7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

1. Kode program Arduino IDE

Saya menyiapkan program pada Arduino IDE untuk menghubungkan NodeMCU ESP8266 ke jaringan Wi-Fi dan broker MQTT lokal. Program ini juga mengatur pembacaan sensor DHT,

pengiriman data suhu dan kelembapan, serta penerimaan perintah untuk mengendalikan LED. Data sensor dikirim secara berkala dalam format JSON melalui topik telemetri, sedangkan perintah LED diterima melalui topik kendali.



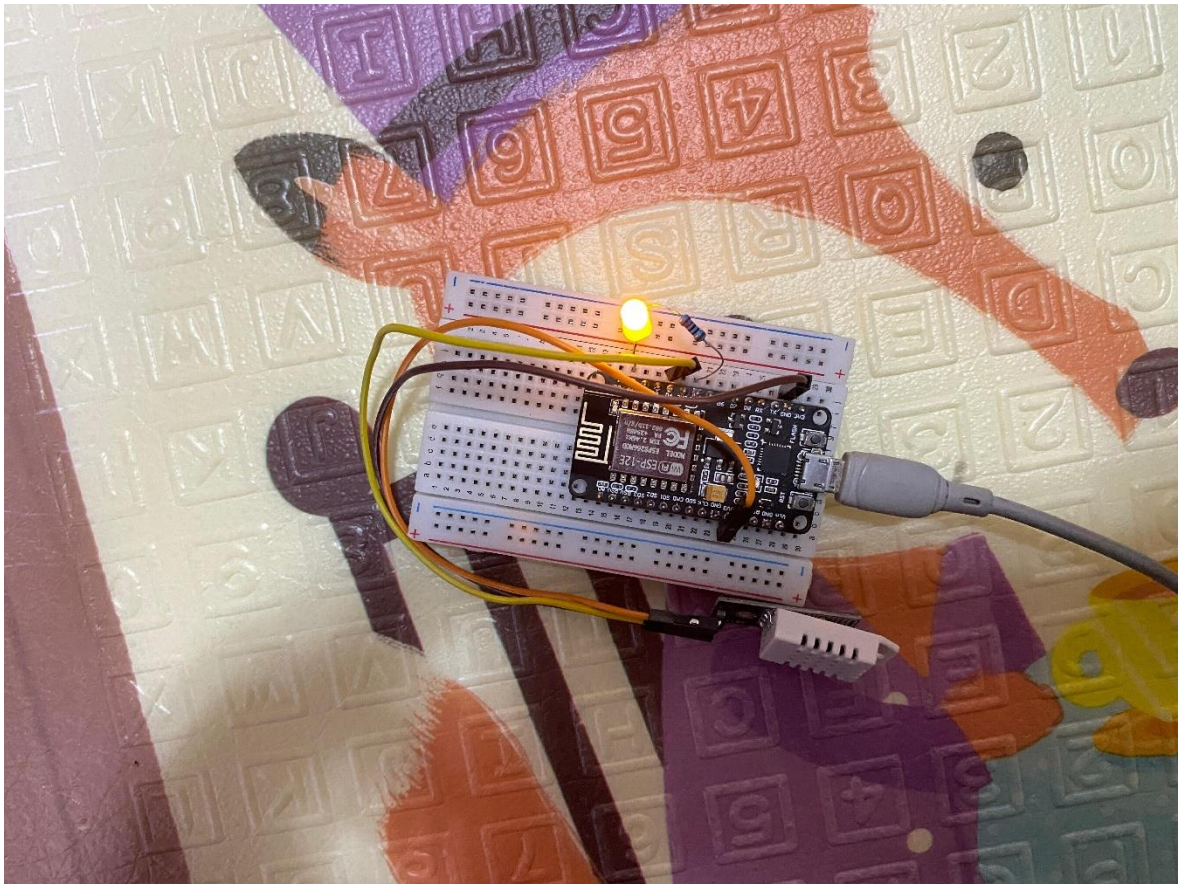






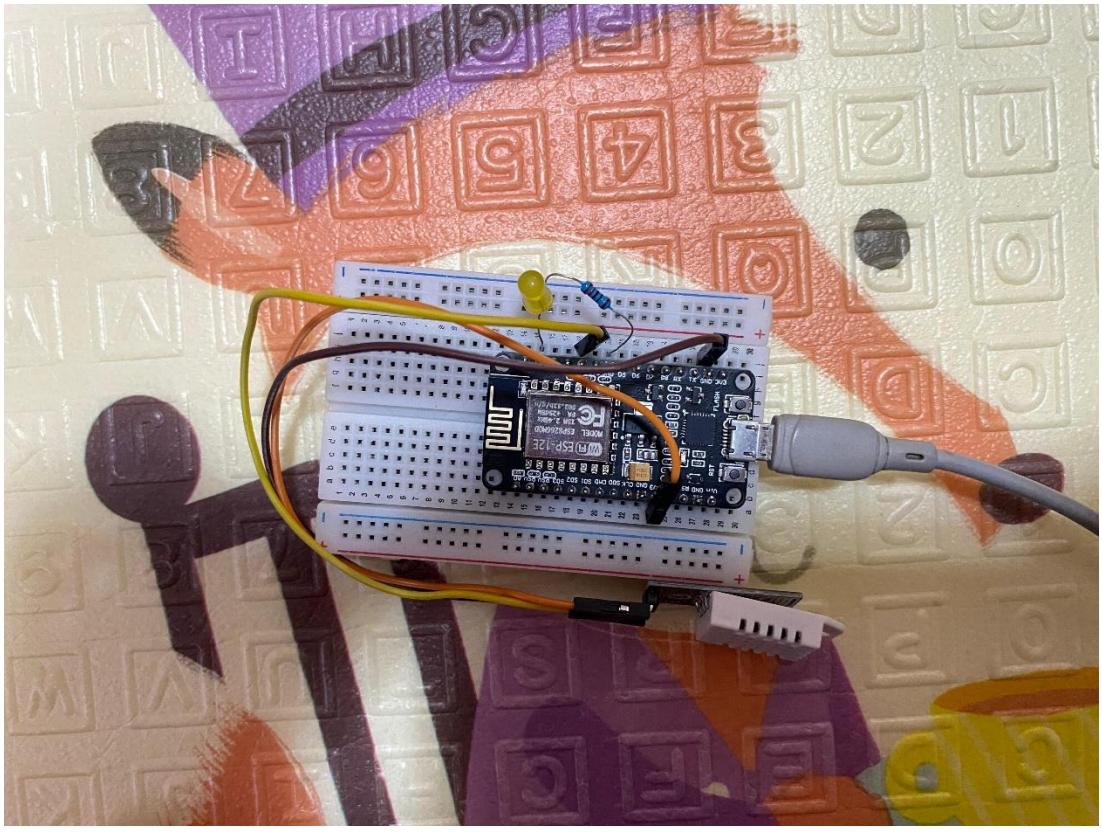
2. Rangkaian fisik NodeMCU dengan LED menyala (ON)

Rangkaian praktikum terdiri dari NodeMCU ESP8266, sensor DHT, breadboard, dan LED indikator. Komponen dirangkai sesuai dengan pin yang digunakan dalam program. Ketika LED menyala, rangkaian menunjukkan kondisi indikator aktif. Rangkaian ini digunakan untuk menguji pembacaan sensor sekaligus pengendalian LED melalui komunikasi MQTT.



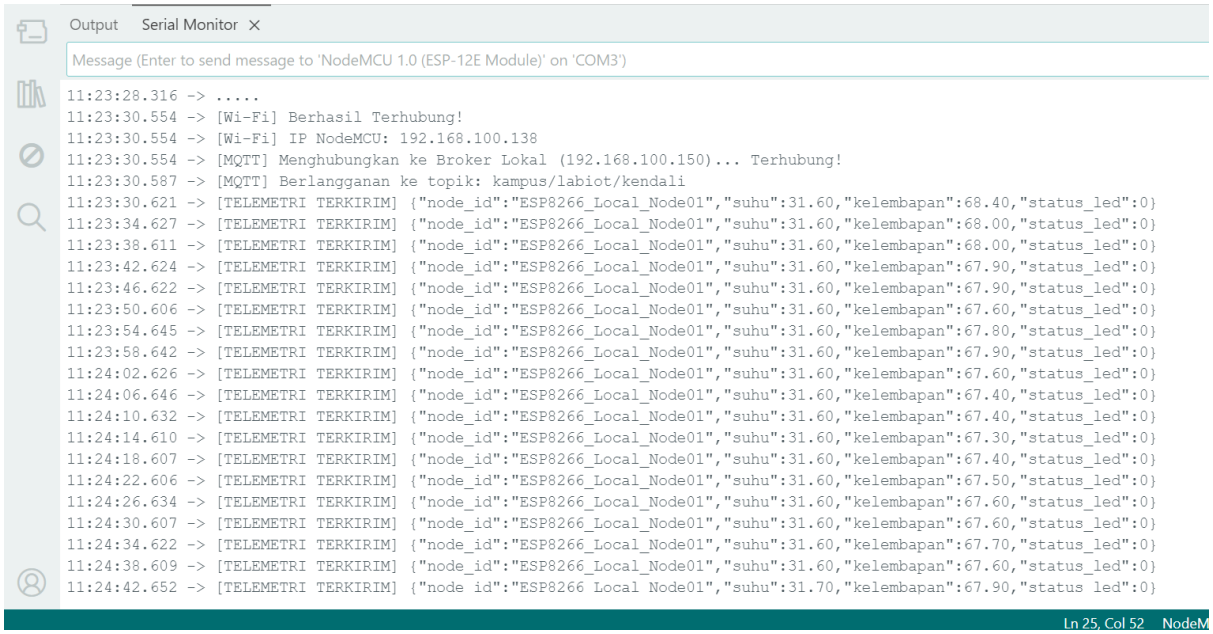
3. Rangkaian fisik ketika LED padam (OFF)

Pada pengujian berikutnya, LED berada dalam kondisi padam, sedangkan NodeMCU tetap mendapatkan daya. Kondisi ini digunakan untuk menunjukkan perbedaan keadaan indikator ketika menerima perintah ON dan OFF. Dengan begitu, perubahan status LED dapat diamati secara langsung pada rangkaian fisik.



4. Hasil Serial Monitor Arduino IDE

Serial Monitor menampilkan proses koneksi NodeMCU ke Wi-Fi dan broker MQTT lokal. Setelah koneksi berhasil, program mengirimkan data telemetri yang berisi identitas perangkat, suhu, kelembapan, dan status LED. Data tersebut muncul secara berkala dalam format JSON. Hasil ini menunjukkan bahwa program dapat membaca sensor dan mengirimkan data melalui broker MQTT lokal.



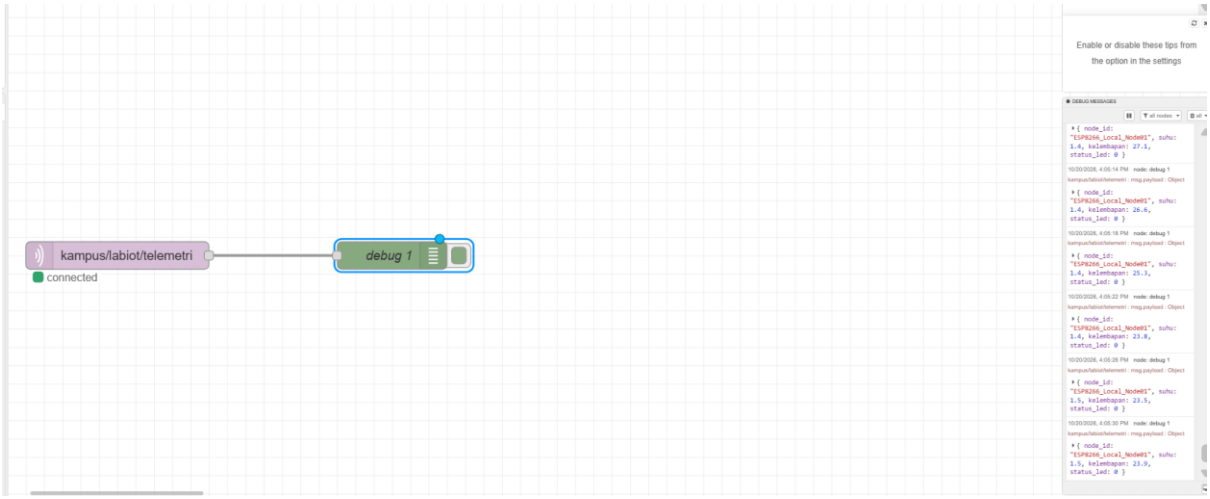
5. Pemeriksaan Layanan Docker dan Broker MQTT

Saya memeriksa layanan Docker untuk memastikan broker MQTT dan Node-RED berjalan. Pemeriksaan ini penting karena NodeMCU membutuhkan broker yang aktif agar data sensor dapat dikirim dan perintah LED dapat diterima. Jika broker berhenti atau koneksi tidak sesuai, komunikasi antara NodeMCU dan Node-RED tidak dapat berjalan dengan baik.



6. Tampilan Node-RED untuk Menerima Data Telemetri

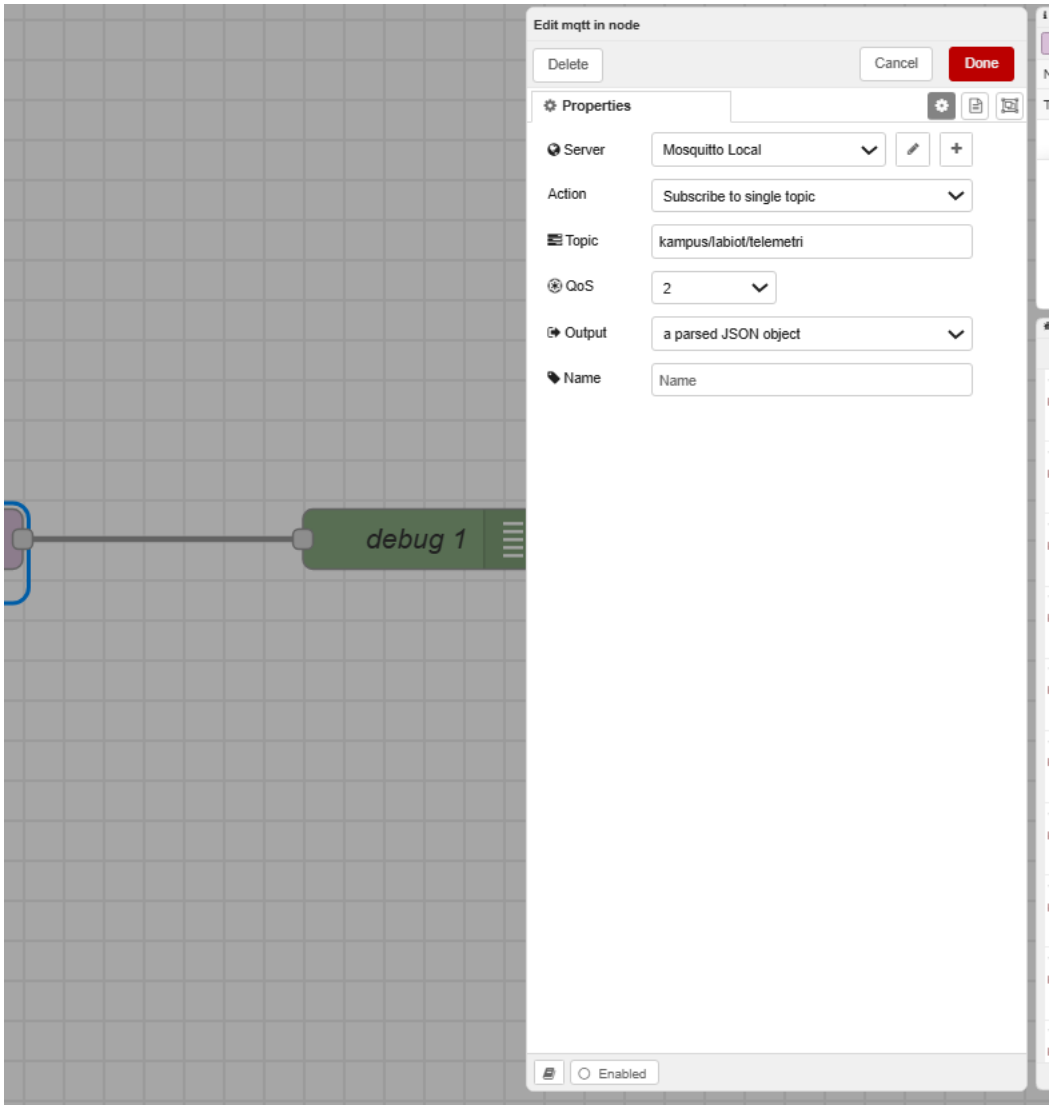
Pada tahap ini, saya membuka Node-RED untuk mempersiapkan penerimaan data dari NodeMCU. Node MQTT In digunakan untuk menerima pesan dari topik telemetri, sedangkan node Debug digunakan untuk menampilkan pesan yang masuk. Tampilan ini menjadi dasar untuk memeriksa apakah data sensor yang dikirim oleh NodeMCU dapat diterima melalui broker lokal.



7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

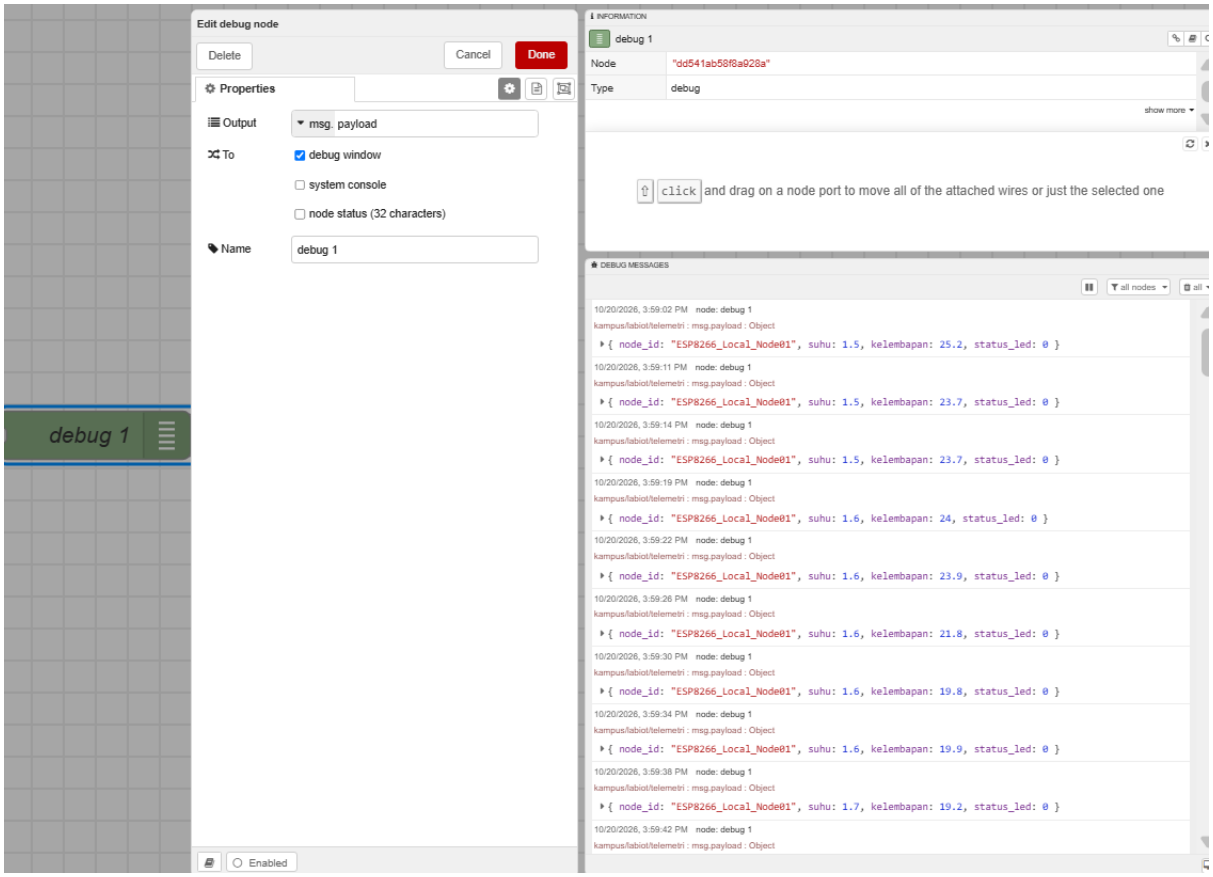
1. Flow dasar Node-RED untuk menerima telemetri

Saya membuat alur sederhana menggunakan node mqtt in dan debug. Node mqtt in menerima data dari topik kampus/labiot/telemetri, kemudian data diteruskan ke debug agar dapat dilihat melalui panel Debug Messages. Saya mengatur server broker menggunakan konfigurasi Mosquitto Local, kemudian menentukan topik kampus/labiot/telemetri. Output dipilih sebagai objek JSON yang sudah diurai (parsed JSON object), sehingga data sensor dapat dibaca sebagai properti seperti suhu dan kelembapan.



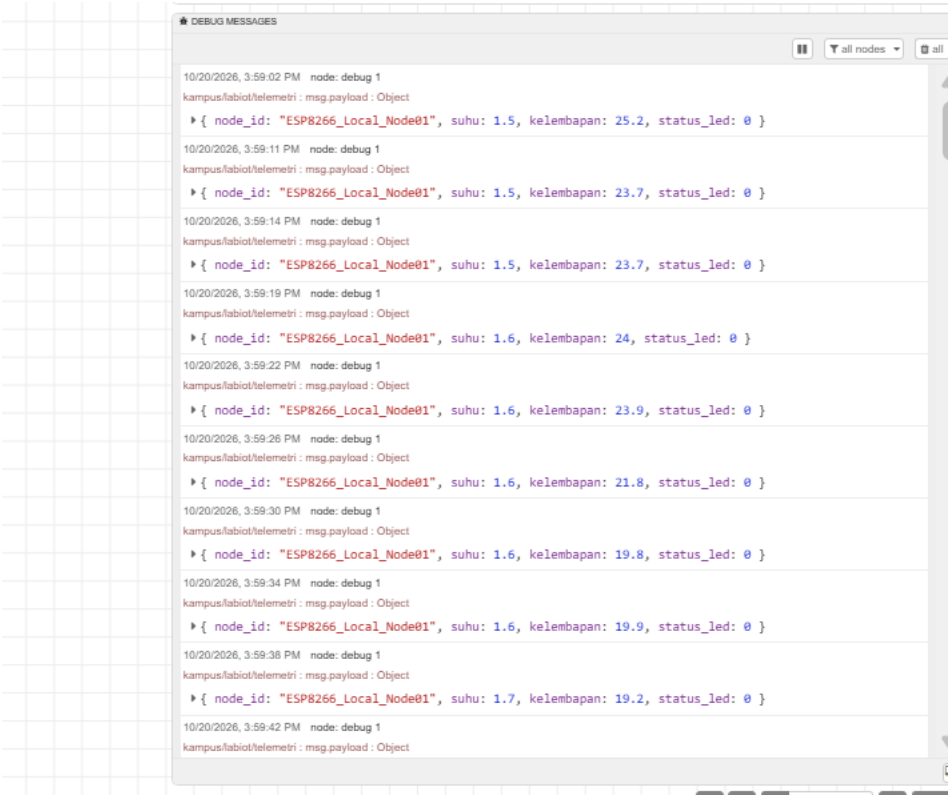
2. Konfigurasi node Debug

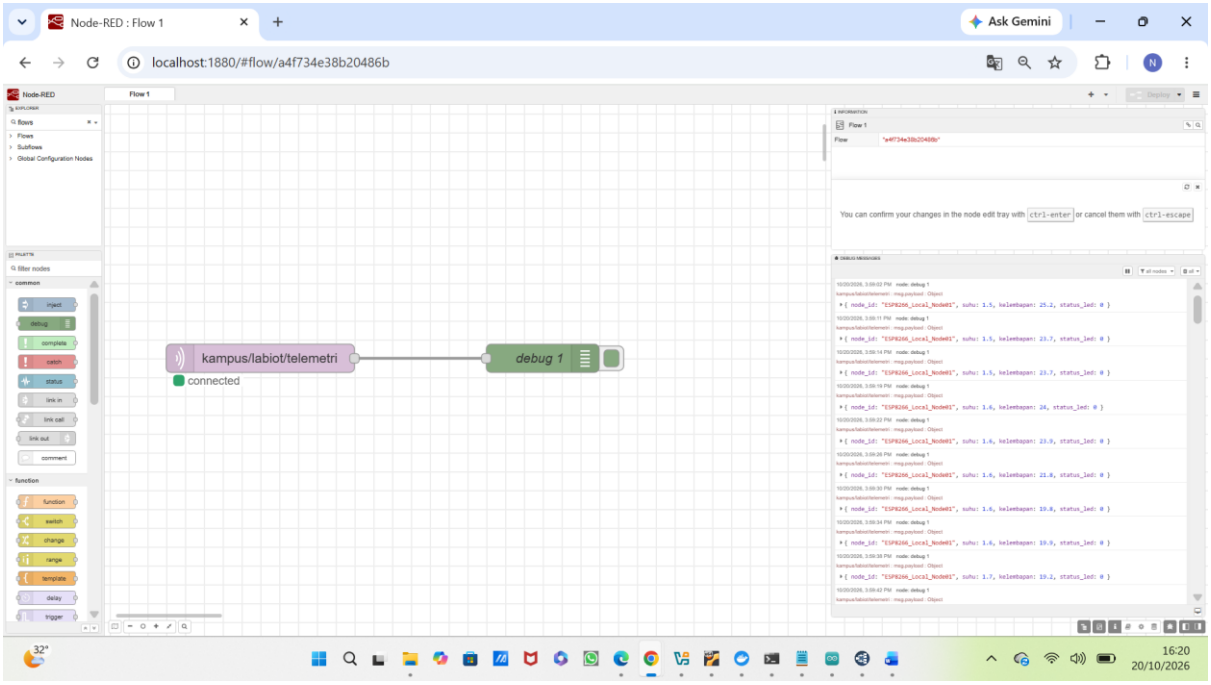
Saya mengatur node Debug agar menampilkan nilai msg.payload. Pengaturan ini memungkinkan isi pesan yang diterima dari MQTT ditampilkan pada panel Debug Messages. Dengan demikian, saya dapat memeriksa data yang masuk tanpa harus selalu melihat Serial Monitor pada Arduino IDE.



3. Data telemetri pada Debug Messages

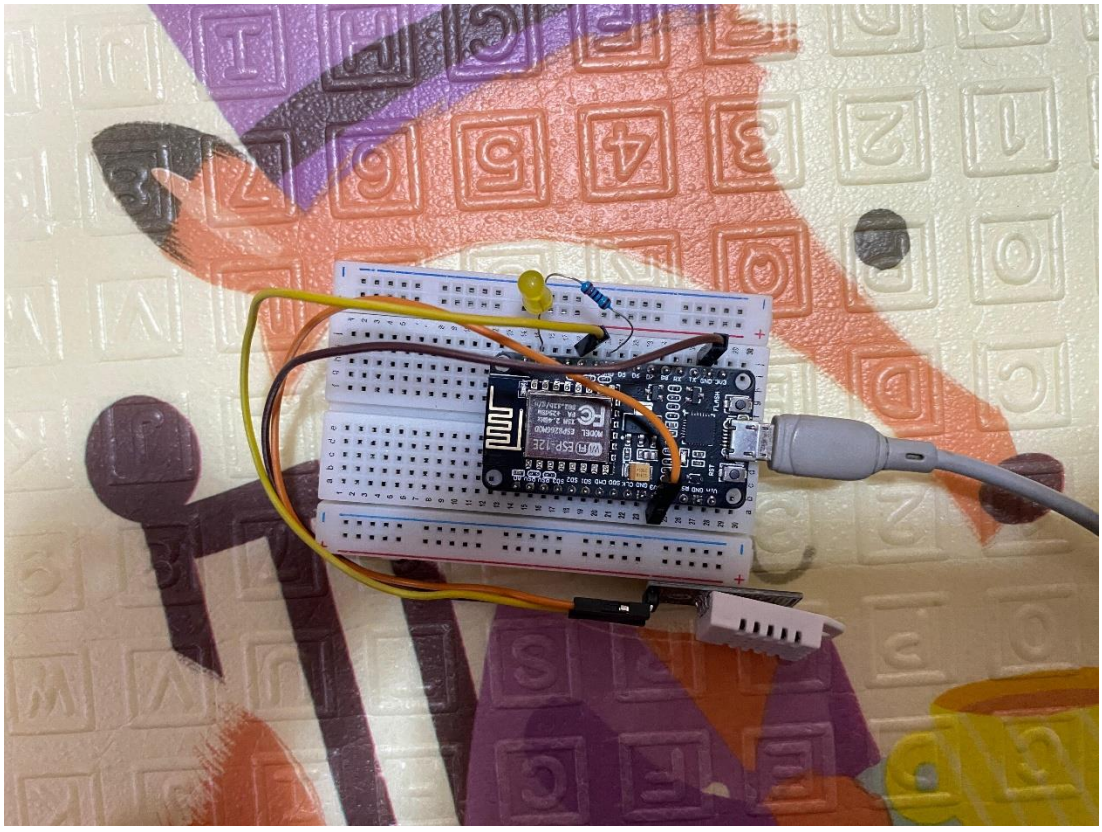
Panel Debug Messages menampilkan data yang diterima dari NodeMCU. Data tersebut memuat node_id, suhu, kelembapan, dan status_led. Dari hasil ini, saya dapat memastikan bahwa data sensor telah diterima dan ditampilkan oleh Node-RED.





4. Rangkaian Fisik

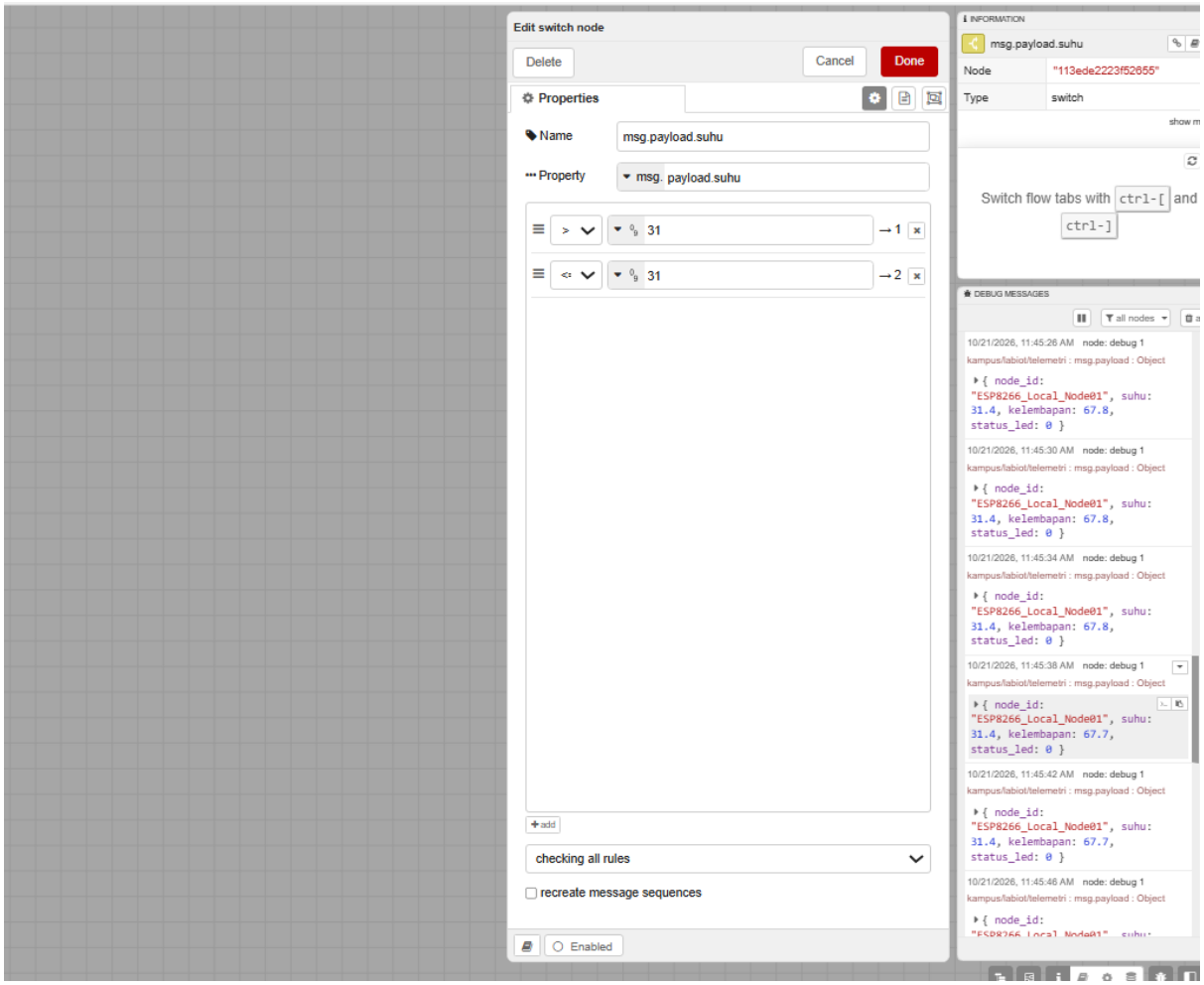
Rangkaian fisik memperlihatkan perangkat yang digunakan untuk menjalankan program, yaitu NodeMCU ESP8266, sensor DHT, dan LED indikator. Sensor digunakan untuk membaca suhu serta kelembapan, sedangkan LED berfungsi sebagai indikator yang dapat dikendalikan. Rangkaian ini menjadi sumber data dan perangkat keluaran yang digunakan dalam pengujian komunikasi MQTT.



7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

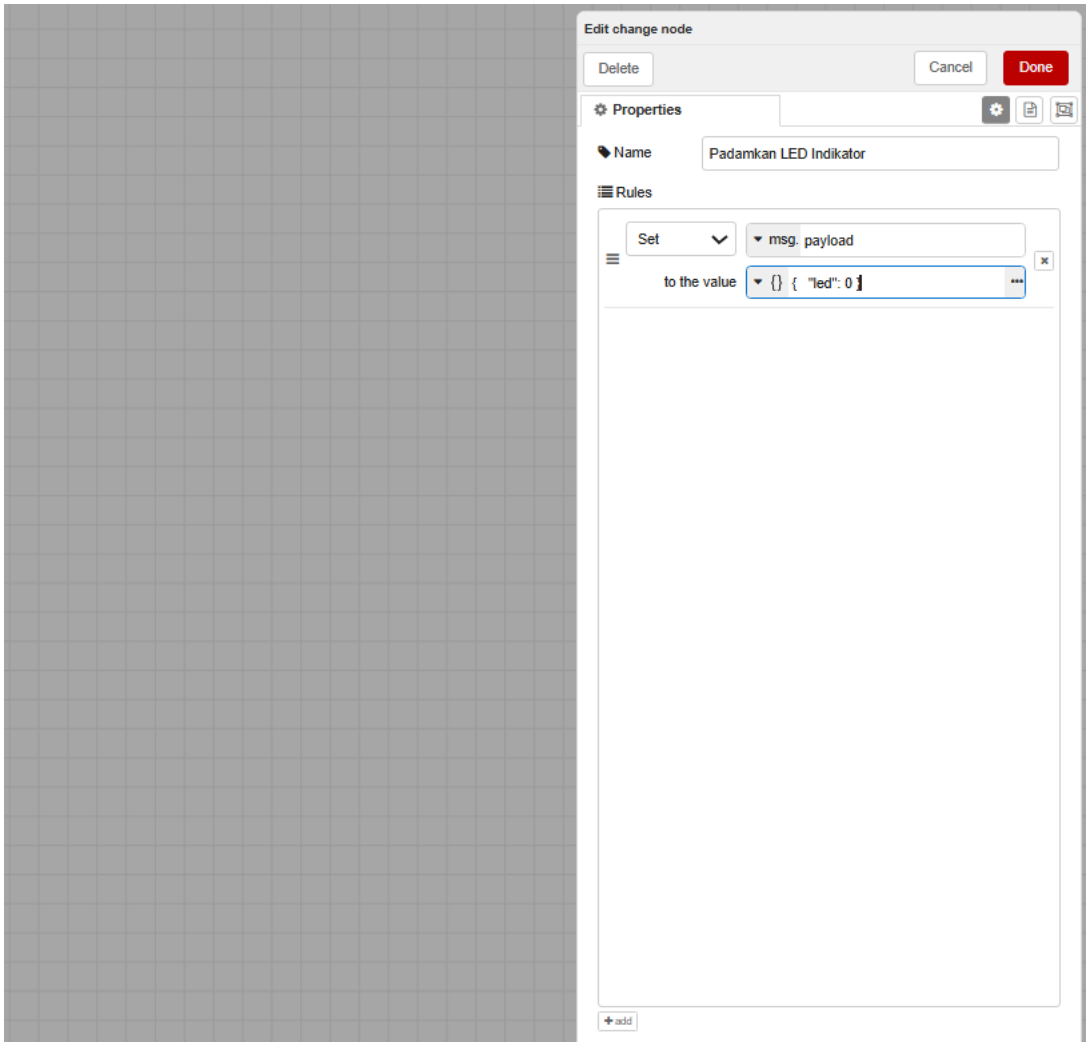
1. Konfigurasi node Switch

Saya mengatur node Switch untuk memeriksa nilai suhu yang diterima dari data telemetri melalui properti msg.payload.suhu. Aturan yang digunakan adalah suhu di atas 31°C untuk menyalakan LED, sedangkan suhu 31°C atau kurang digunakan untuk memadamkan LED. Node Switch menentukan jalur pesan yang sesuai berdasarkan hasil pembacaan suhu dari sensor.



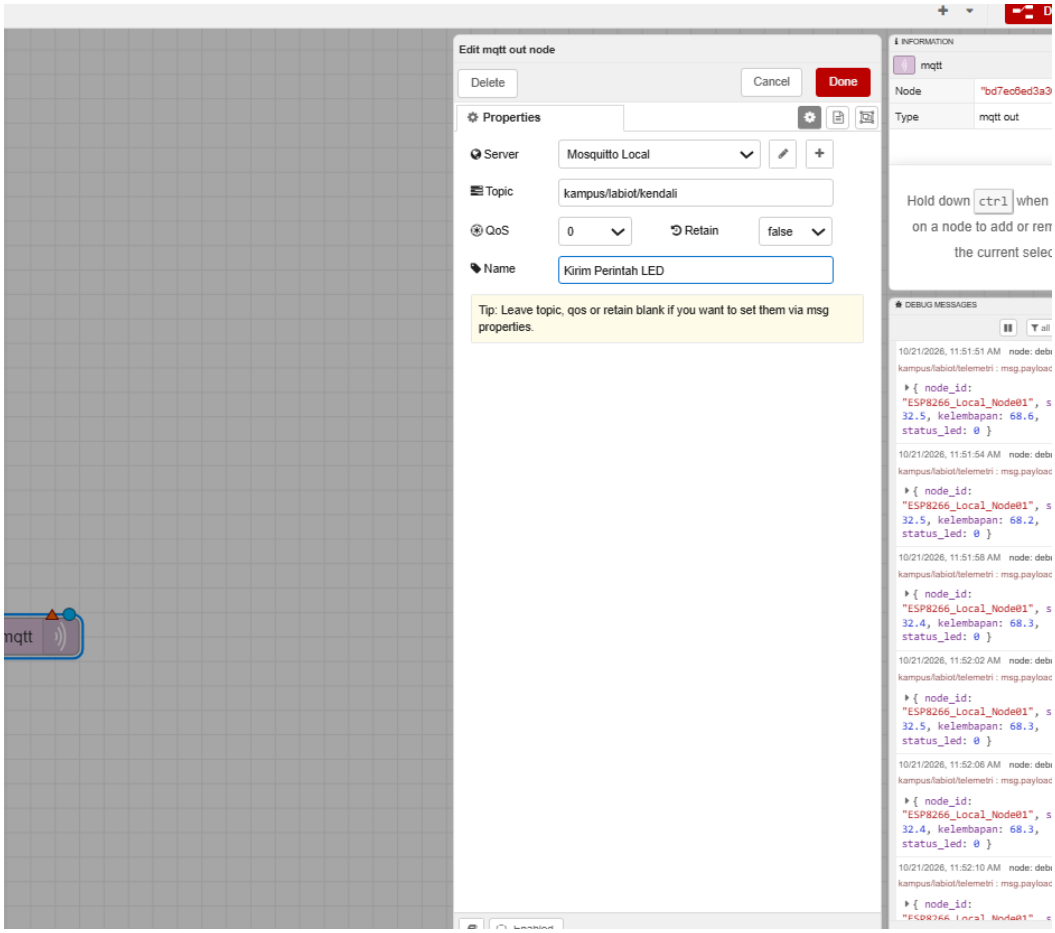
2. Konfigurasi node Change untuk mematikan LED

Pada tahap ini, saya mengatur node Change bernama Padamkan LED Indikator untuk mengubah isi pesan menjadi perintah JSON {"led":0}. Perintah tersebut digunakan ketika suhu berada pada kondisi yang ditetapkan untuk memadamkan LED. Pesan ini kemudian diteruskan ke node MQTT Out agar dapat dikirimkan kepada NodeMCU melalui broker MQTT.



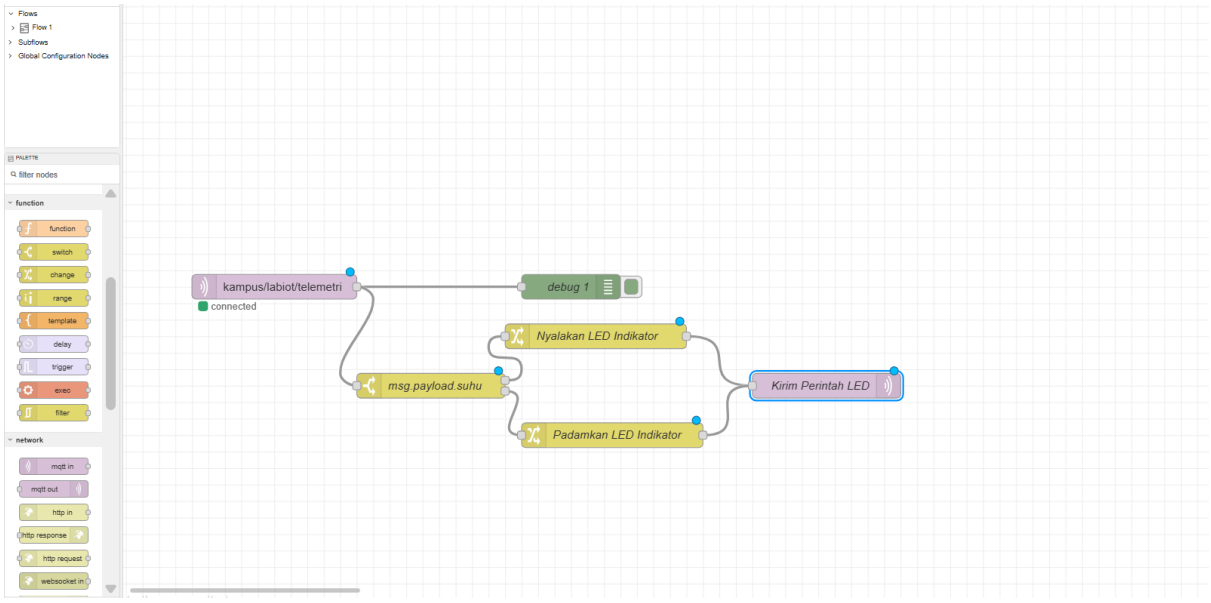
3. Konfigurasi node MQTT Out

Saya mengatur node MQTT Out agar mengirimkan perintah kendali melalui topik kampus/labiot/kendali. Topik ini digunakan untuk menyampaikan instruksi dari Node-RED ke NodeMCU. Dengan konfigurasi tersebut, perintah yang dihasilkan oleh aturan suhu dapat diteruskan melalui broker MQTT dan diproses oleh perangkat.



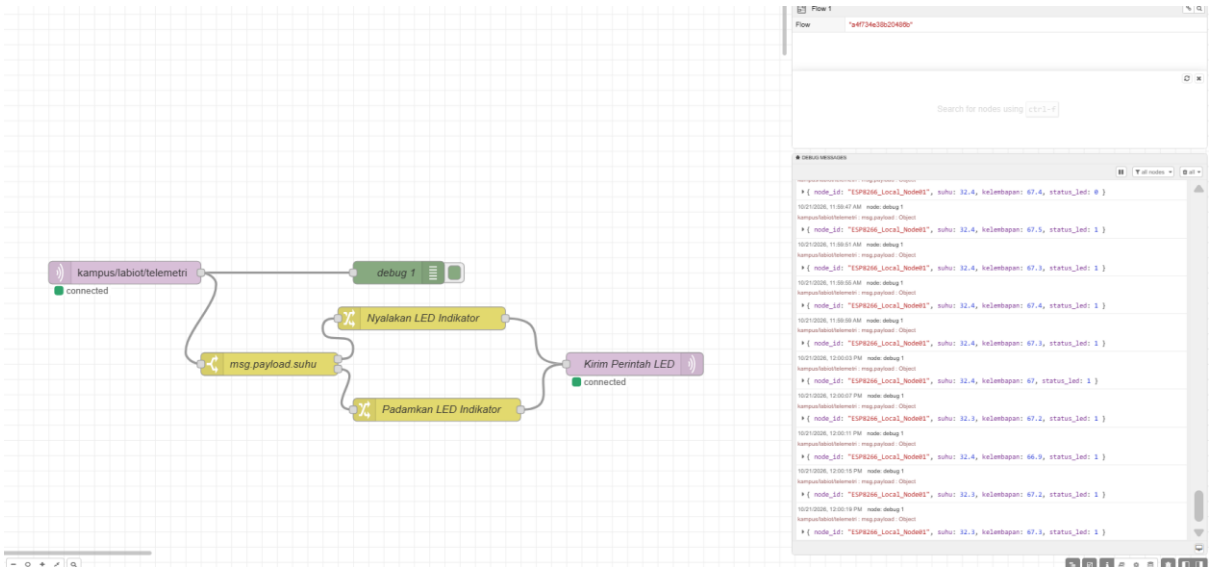
4. Flow lengkap Rule Engine Praktikum 4

Setelah setiap node selesai dikonfigurasi, saya menyusun flow yang menghubungkan MQTT In, Switch, dua node Change, dan MQTT Out. Data suhu yang diterima akan diperiksa oleh node Switch. Jika suhu lebih dari 31°C, sistem menyiapkan perintah {"led":1} untuk menyalakan LED. Sebaliknya, jika suhu 31°C atau kurang, sistem menyiapkan perintah {"led":0} untuk memadamkan LED. Alur ini membuat pengendalian LED dapat dilakukan secara otomatis berdasarkan data sensor.



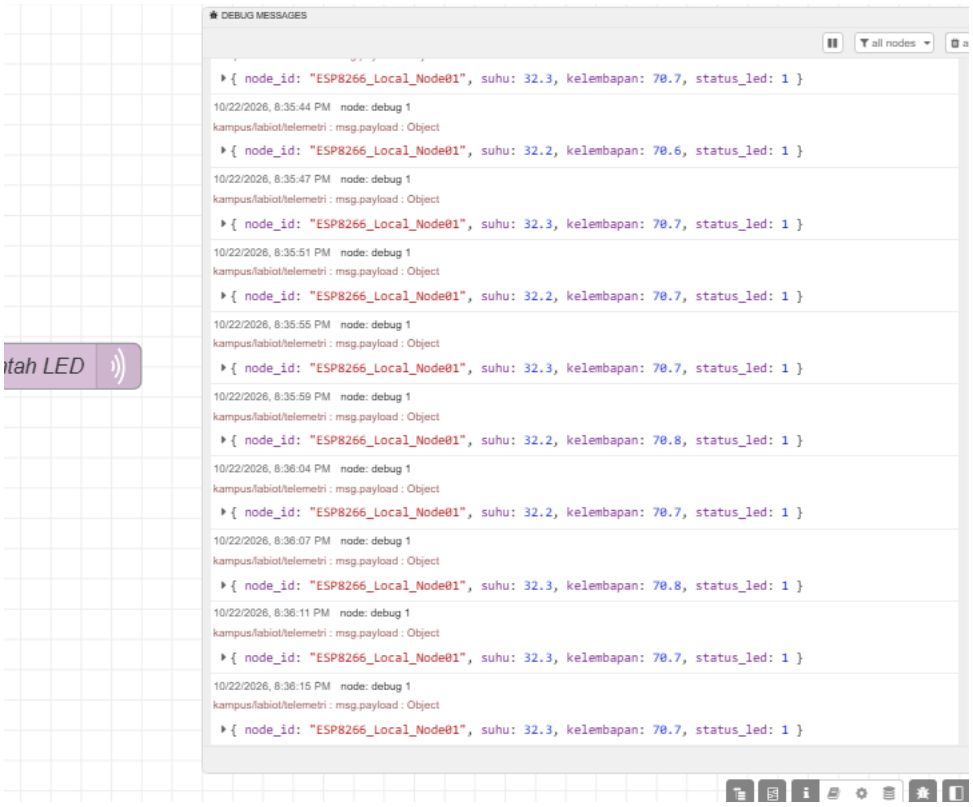
5. Pengujian flow saat LED menyala

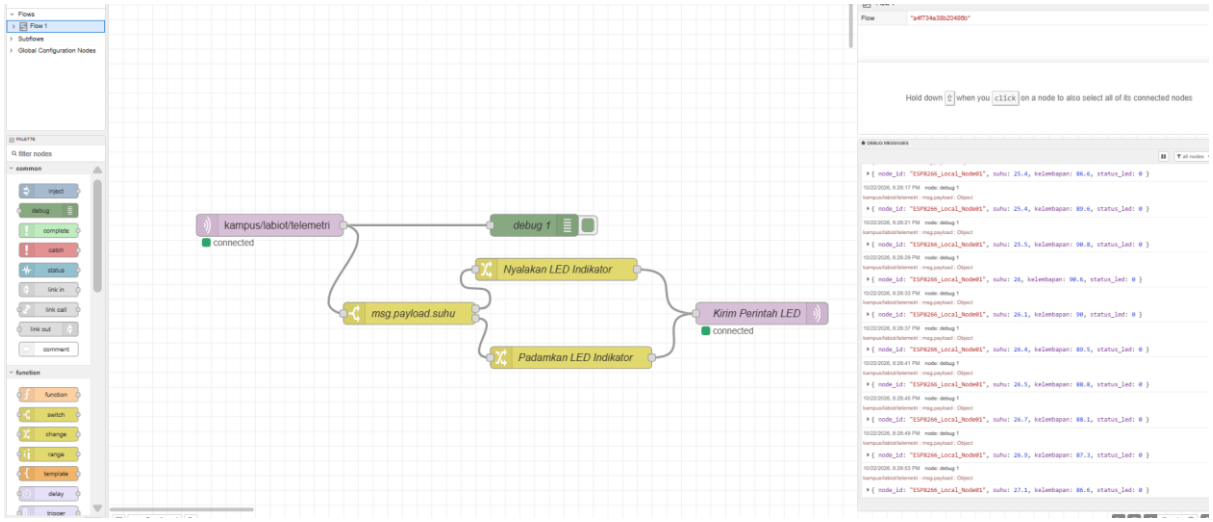
Saya melakukan pengujian untuk melihat apakah aturan suhu dapat memicu perintah menyalakan LED. Ketika suhu yang terbaca lebih dari 31°C, flow memilih jalur untuk menyalakan LED dan menghasilkan pesan {"led":1}. Pesan tersebut dikirim melalui topik kendali MQTT menuju NodeMCU. Hasil ini menunjukkan bahwa aturan untuk menyalakan LED telah diterapkan pada flow Node-RED.



6. Hasil Debug Messages saat pengujian

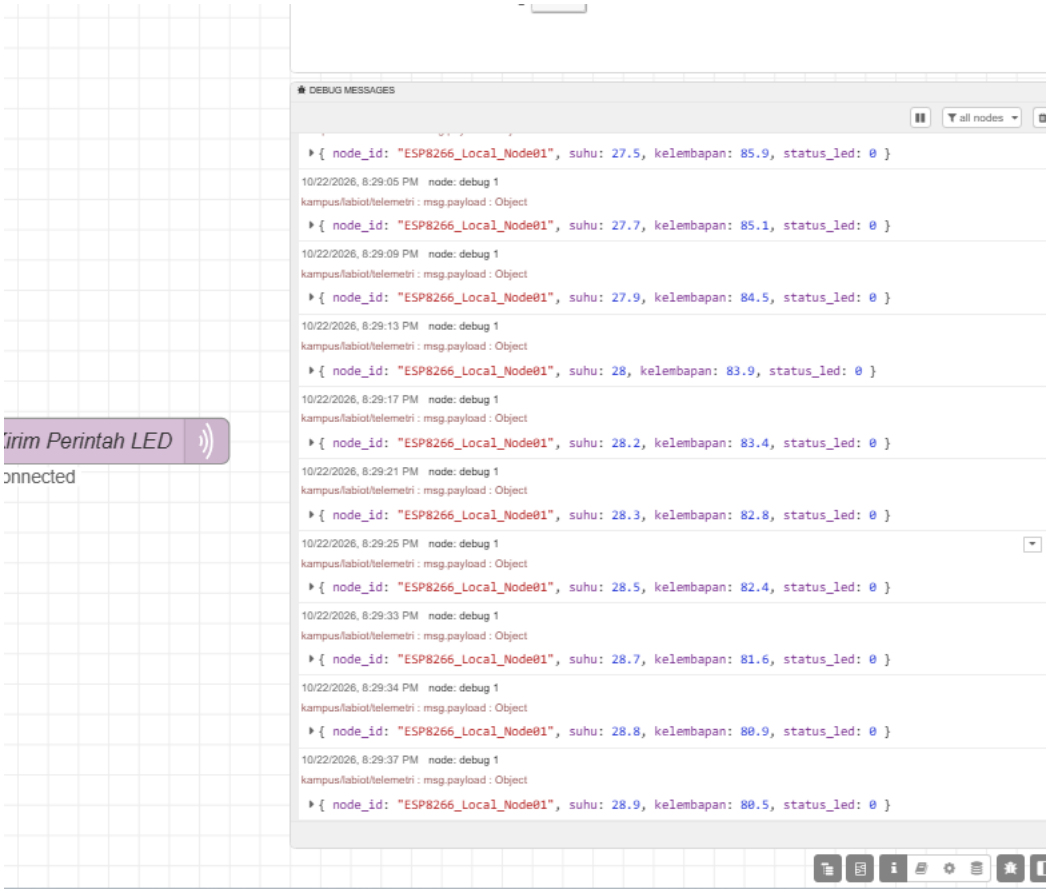
Panel Debug Messages digunakan untuk memeriksa data yang masuk dan pesan yang dihasilkan oleh flow. Data telemetri yang diterima memuat informasi suhu, kelembapan, dan status LED. Jika node Debug dihubungkan ke keluaran Function atau Change yang menghasilkan perintah kendali, panel juga dapat menampilkan pesan seperti {"led":1} atau {"led":0}. Dengan begitu, saya dapat memeriksa proses pengolahan pesan sebelum perintah diteruskan ke NodeMCU.





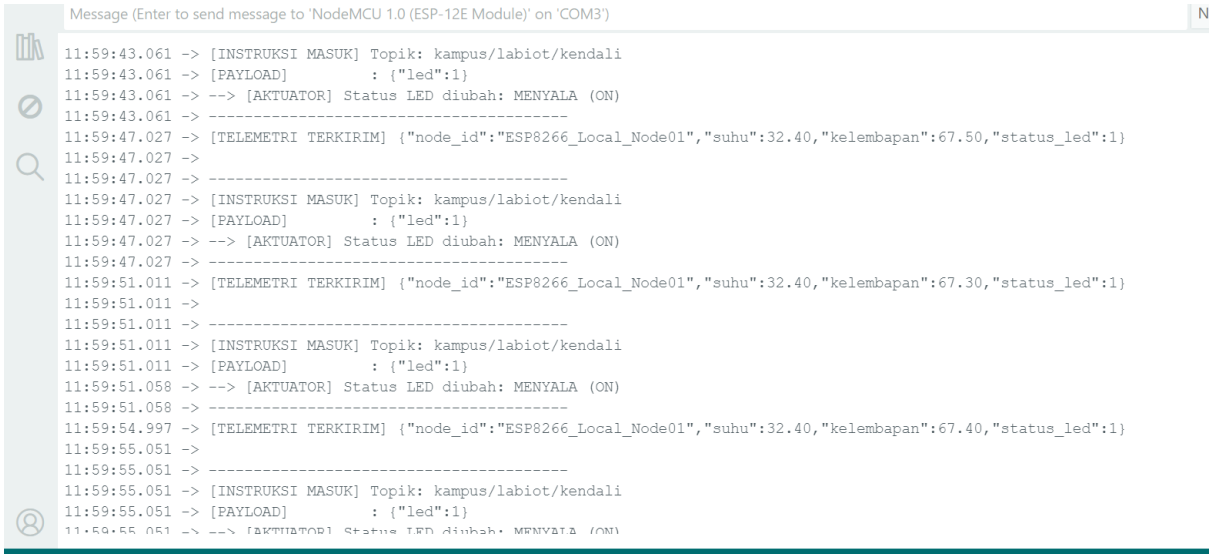
7. Pengujian Flow Saat LED Padam

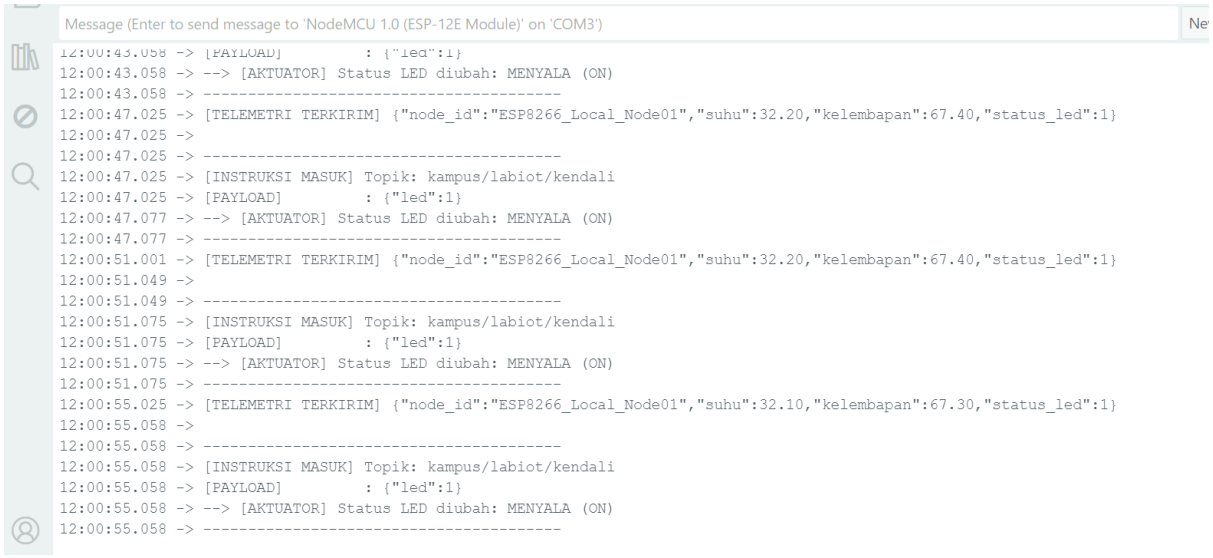
Pada pengujian ini, LED indikator berada dalam kondisi padam setelah NodeMCU menerima perintah {"led": 0} dari Node-RED melalui broker MQTT. Perintah tersebut dihasilkan ketika kondisi suhu memenuhi aturan pemadaman yang sudah ditentukan. Setelah pesan diterima, program Arduino mengatur pin LED ke kondisi LOW sehingga lampu mati. Dari hasil ini, saya dapat melihat bahwa perintah yang dibuat melalui Node-RED berhasil diteruskan ke perangkat dan dijalankan sesuai dengan aturan yang digunakan.



8. Hasil Serial Monitor Saat LED Menyala

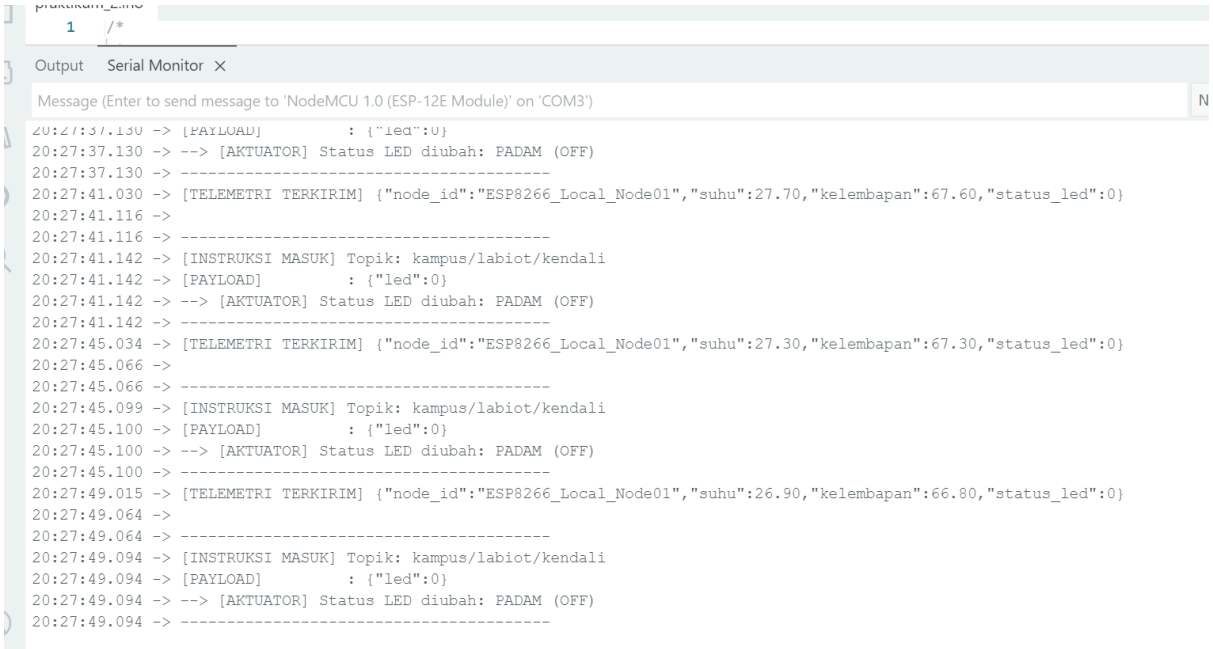
Serial Monitor digunakan untuk melihat pesan yang diterima NodeMCU dari topik kendali MQTT. Ketika perintah {"led":1} diterima, program mengubah keluaran LED menjadi menyala dan menampilkan keterangan bahwa LED berada dalam kondisi ON. Sementara itu, ketika perintah {"led":0} diterima, LED dipadamkan dan Serial Monitor menampilkan status OFF. Hasil ini membantu saya memastikan bahwa perintah dari Node-RED tidak hanya dibuat, tetapi juga diterima dan diproses oleh perangkat.





9. Pengujian Ketika Suhu Berada di Bawah atau Sama dengan 31°C

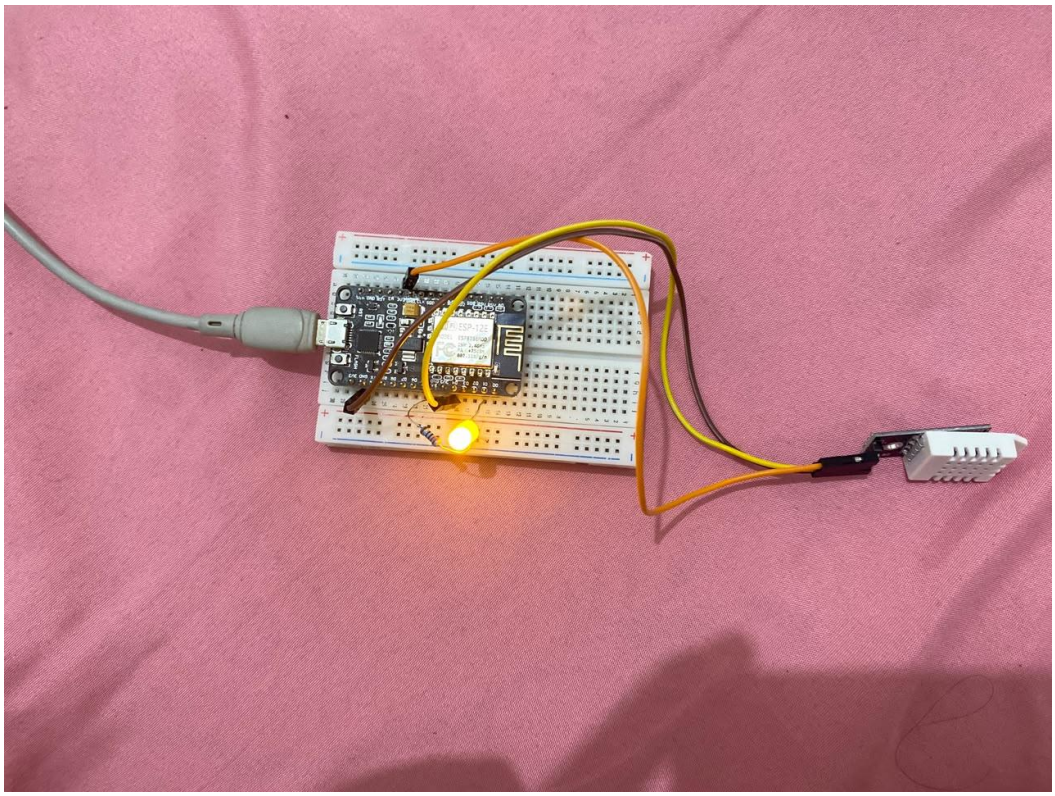
Saya mengamati kondisi ketika suhu yang dibaca sensor berada pada nilai 31°C atau lebih rendah. Berdasarkan aturan yang digunakan pada Praktikum 4, sistem memilih jalur untuk memadamkan LED dan mengirimkan perintah {"led":0} melalui broker MQTT. Jika pesan tersebut diterima oleh NodeMCU, LED akan berubah menjadi padam. Pengujian ini digunakan untuk memeriksa apakah aturan pemadaman LED berjalan sesuai dengan batas suhu yang sudah ditetapkan.



10. Rangkaian Fisik Praktikum 4 Ketika LED Menyala

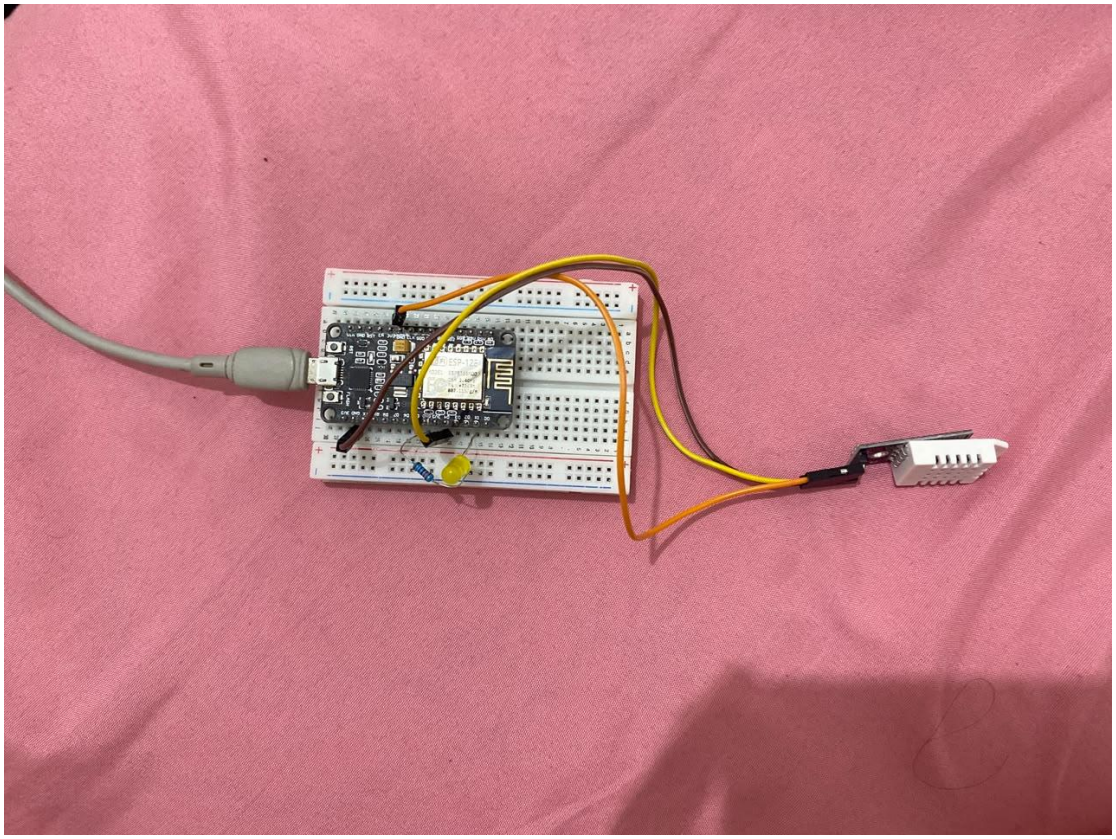
Rangkaian fisik memperlihatkan kondisi LED ketika perintah untuk menyalakannya berhasil dijalankan. NodeMCU menerima perintah kendali dari broker MQTT, kemudian mengatur pin LED agar berada dalam kondisi ON. Hasil ini menunjukkan bahwa proses pengendalian

tidak hanya terlihat pada flow Node-RED, tetapi juga menghasilkan perubahan pada perangkat fisik.



11. Rangkaian Fisik Praktikum 4 Ketika LED Padam (OFF)

Pada pengujian ini, LED indikator terlihat padam setelah perangkat menerima perintah OFF. Rangkaian yang digunakan tetap sama, tetapi status LED berubah sesuai dengan instruksi yang dikirim melalui MQTT. Hasil tersebut menunjukkan bahwa sistem dapat mengendalikan LED dalam dua kondisi, yaitu menyala dan padam, berdasarkan aturan suhu yang diterapkan.



7.6 Latihan

Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):
Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Jawab

Menurut saya, penggunaan localhost pada Node-RED bisa menyebabkan koneksi gagal karena localhost mengarah ke container Node-RED itu sendiri, bukan ke container Mosquitto. Padahal, broker MQTT berada di container yang berbeda.

Docker Compose menyediakan jaringan internal yang memungkinkan kedua container saling berkomunikasi. Docker juga memiliki DNS internal yang dapat mengenali nama layanan, sehingga Node-RED cukup menggunakan nama layanan mqtt untuk menemukan broker MQTT tanpa harus mengetahui alamat IP container tersebut secara manual.

Jadi, pada konfigurasi Node-RED saya menggunakan alamat server mqtt dengan port 1883. Dengan begitu, Node-RED dapat terhubung ke broker Mosquitto melalui jaringan internal Docker.

Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):
Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

Jawab

koneksi bisa gagal karena ESP8266 dan laptop berada pada subnet yang berbeda. Jika menggunakan subnet mask 255.255.255.0, ESP8266 berada di jaringan 192.168.43.0/24, sedangkan laptop berada di jaringan 192.168.1.0/24. Dalam kondisi praktikum biasa tanpa routing antara kedua jaringan, ESP8266 tidak bisa langsung menjangkau broker MQTT di laptop.

Cara mengatasinya adalah menghubungkan laptop dan ESP8266 ke jaringan Wi-Fi yang sama agar keduanya berada dalam subnet yang sesuai. Setelah itu, alamat broker pada kode Arduino harus disesuaikan dengan IPv4 laptop yang digunakan saat itu.

Saya juga perlu memastikan container Mosquitto berjalan, port TCP 1883 sudah dipublikasikan oleh Docker, dan Windows Firewall mengizinkan koneksi masuk pada port tersebut jika diperlukan. Dengan langkah itu, ESP8266 dapat menjangkau broker MQTT lokal dan mengirimkan data telemetri.

Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas $31.0°C$(misal: $31.05°C \rightarrow 30.95°C \rightarrow 31.02°C$), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan function node berbahasa JavaScript yang menerapkan logika Histeresis:

LED baru menyala jika suhu $> 31.5°C$(msg.payload = {"led": 1}).

LED baru padam jika suhu mendingin hingga $< 30.0°C$(msg.payload = {"led": 0}).
Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

Jawab

histeresis digunakan supaya LED tidak sering berkedip ketika suhu naik dan turun di sekitar batas tertentu. Kalau hanya menggunakan satu batas suhu, LED bisa terus berubah antara menyala dan mati ketika suhu sedikit berubah.

Pada latihan ini, saya menggunakan dua batas suhu. LED akan menyala jika suhu lebih dari 31,5°C dan akan mati jika suhu turun di bawah 30°C. Kalau suhu berada di antara 30°C sampai 31,5°C, status LED akan tetap mengikuti kondisi sebelumnya.

Untuk menerapkan aturan tersebut, saya menggunakan Function Node di Node-RED dengan kode JavaScript berikut. **Kode JavaScript Function Node:**

```
let suhu = Number(msg.payload.suhu);

// Memeriksa apakah data suhu valid

if (!Number.isFinite(suhu)) {

    return null;

}

// Mengambil status LED sebelumnya

let ledState = context.get("ledState");

// Jika belum ada status sebelumnya, LED dimulai dalam keadaan mati

if (ledState === undefined) {

    ledState = 0;

}

// Aturan histeresis

if (suhu > 31.5) {

    ledState = 1;

} else if (suhu < 30.0) {

    ledState = 0;

}

// Menyimpan status LED terakhir

context.set("ledState", ledState);

// Menyiapkan perintah untuk ESP8266

msg.payload = {

    led: ledState

};

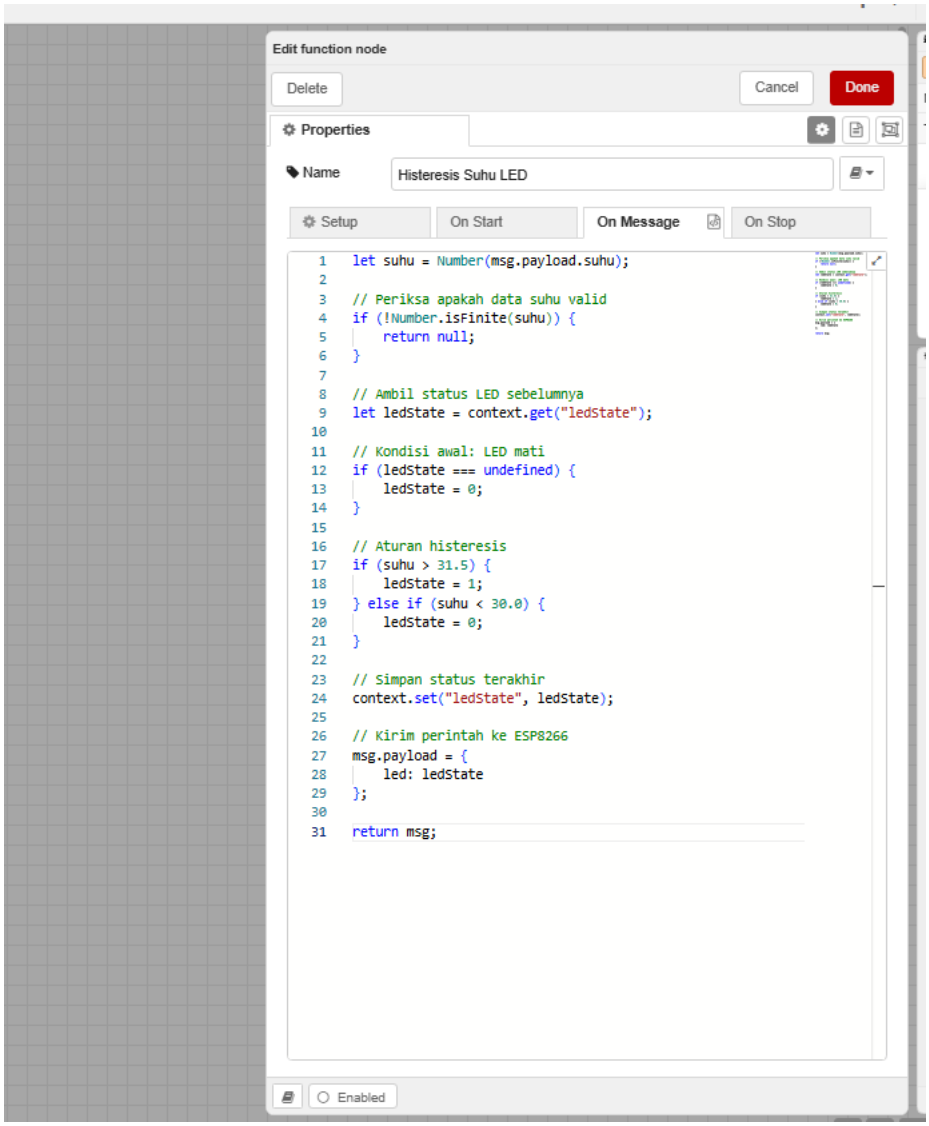
return msg;
```

Dari pengujian yang saya lakukan, ketika suhu lebih dari 31,5°C, LED menyala. Ketika suhu turun ke rentang 30°C sampai 31,5°C, LED tetap berada pada kondisi sebelumnya. Setelah suhu turun di bawah 30°C, LED baru mati.

Dengan menggunakan dua batas suhu tersebut, perubahan status LED menjadi lebih stabil dan tidak mudah berkedip akibat perubahan suhu yang kecil.

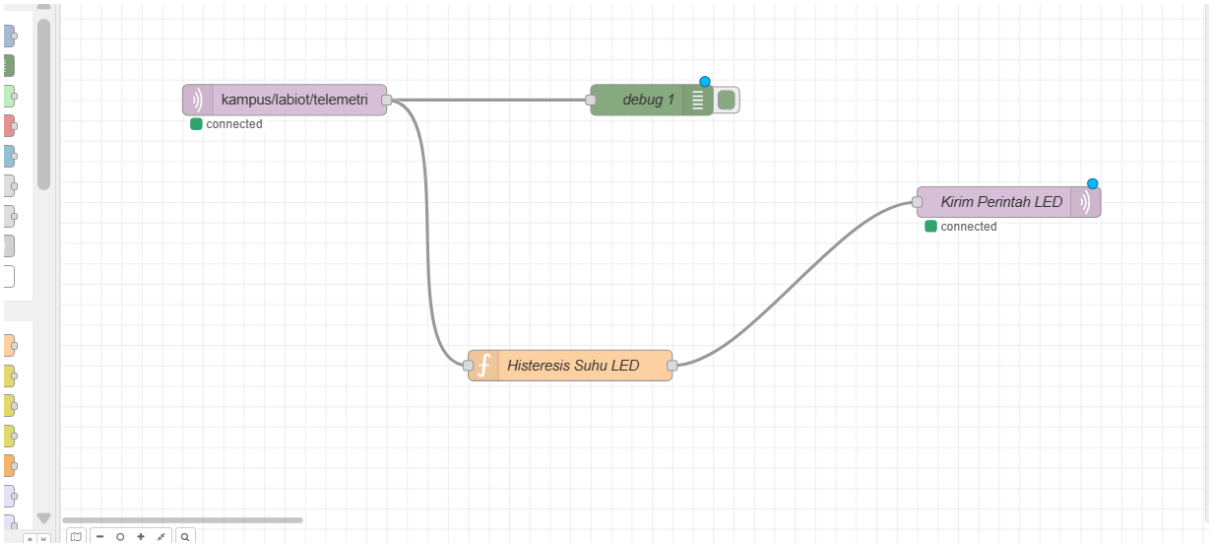
1. Kode JavaScript pada Function Node

Untuk membuat sistem pengendalian LED lebih stabil, saya memasukkan kode JavaScript ke dalam Function Node bernama Histeresis Suhu LED. Kode tersebut membaca nilai suhu dari data telemetri, kemudian memeriksa apakah suhu sudah melewati batas yang ditentukan.
LED akan menyala ketika suhu lebih dari 31,5°C dan baru padam ketika suhu turun di bawah 30°C. Jika suhu masih berada di antara kedua batas tersebut, status LED akan tetap mengikuti keadaan sebelumnya.



2. Flow Node-RED untuk histeresis

Setelah kode dimasukkan, saya menghubungkan node mqtt in ke Function Node agar data suhu dari sensor dapat diproses secara otomatis. Hasil pengolahan kemudian diteruskan ke node mqtt out dengan topik kampus/labiot/kendali. Alur ini membuat keputusan menyalakan atau mematikan LED dilakukan oleh Function Node berdasarkan suhu yang diterima, bukan dengan mengatur LED secara manual setiap kali suhu berubah.



3. Pengujian LED ON saat suhu mencapai 31,60°C

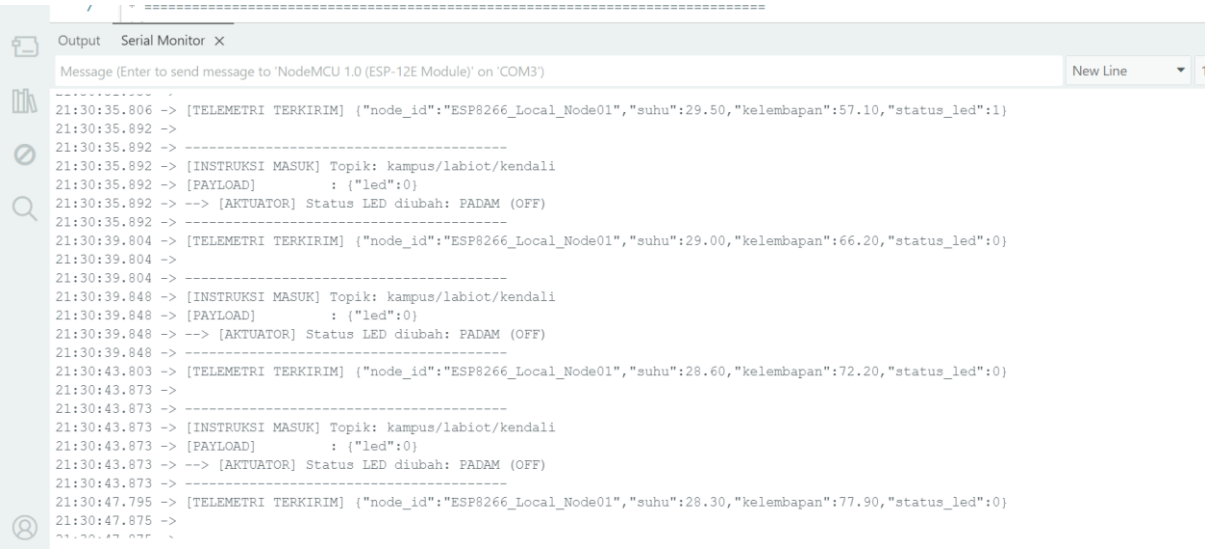
Saya melakukan pengujian ketika suhu yang terbaca mencapai 31,60°C. Nilai ini sudah melewati batas 31,5°C, sehingga Function Node menghasilkan perintah {"led":1} untuk menyalakan LED. Hasil pengujian tersebut menunjukkan bahwa aturan untuk menyalakan LED berjalan sesuai dengan kondisi yang sudah ditetapkan pada kode histeresis.

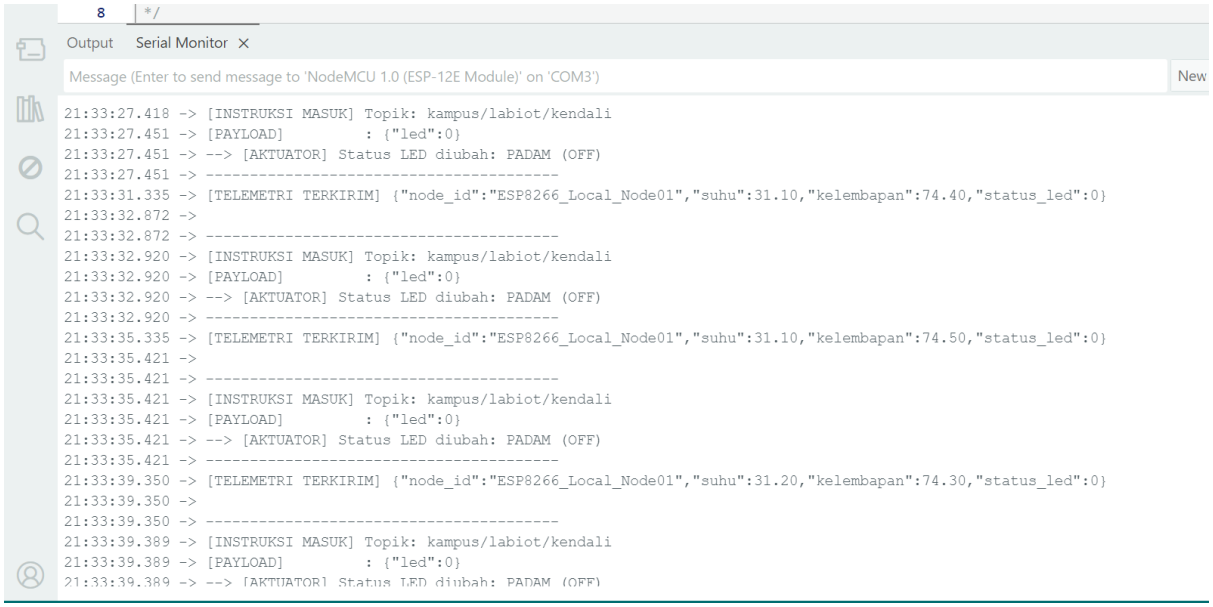


4. Pengujian LED OFF ketika suhu turun di bawah 30°C

Berikutnya, saya mengamati perubahan kondisi ketika suhu sensor turun hingga di bawah 30°C. Pada kondisi tersebut, sistem menghasilkan perintah {"led":0} yang dikirimkan melalui broker MQTT ke NodeMCU. Setelah perintah diterima, LED berubah menjadi

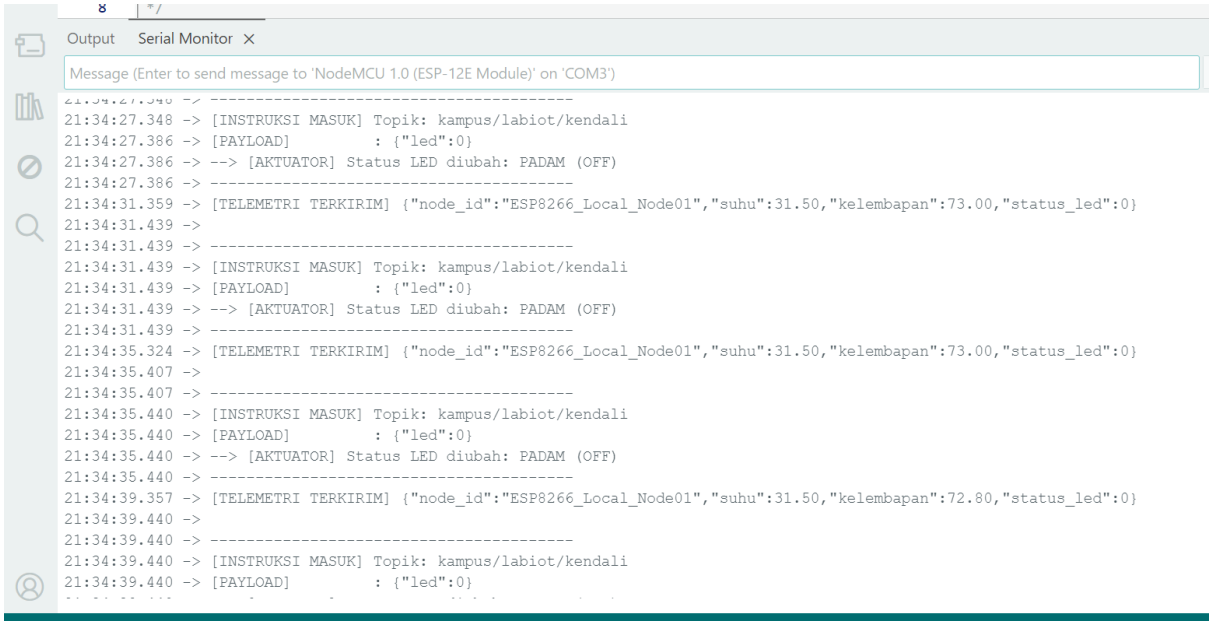
padam. Hasil ini menunjukkan bahwa aturan untuk mematikan LED juga berjalan sesuai dengan batas suhu yang sudah ditentukan.


Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line
21:30:35.806 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":29.50,"kelembapan":57.10,"status_led":1}
21:30:35.892 ->
21:30:35.892 -> ----------
21:30:35.892 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
21:30:35.892 -> [PAYLOAD] : {"led":0}
21:30:35.892 -> --> [AKTUATOR] Status LED diubah: PADAM (OFF)
21:30:35.892 -> ----------
21:30:39.804 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":29.00,"kelembapan":66.20,"status_led":0}
21:30:39.804 ->
21:30:39.804 -> ----------
21:30:39.848 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
21:30:39.848 -> [PAYLOAD] : {"led":0}
21:30:39.848 -> --> [AKTUATOR] Status LED diubah: PADAM (OFF)
21:30:39.848 -> ----------
21:30:43.803 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":72.20,"status_led":0}
21:30:43.873 ->
21:30:43.873 -> ----------
21:30:43.873 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
21:30:43.873 -> [PAYLOAD] : {"led":0}
21:30:43.873 -> --> [AKTUATOR] Status LED diubah: PADAM (OFF)
21:30:43.873 -> ----------
21:30:47.795 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.30,"kelembapan":77.90,"status_led":0}
21:30:47.875 ->
21:30:47.875 ->
21:30:47.875 ->

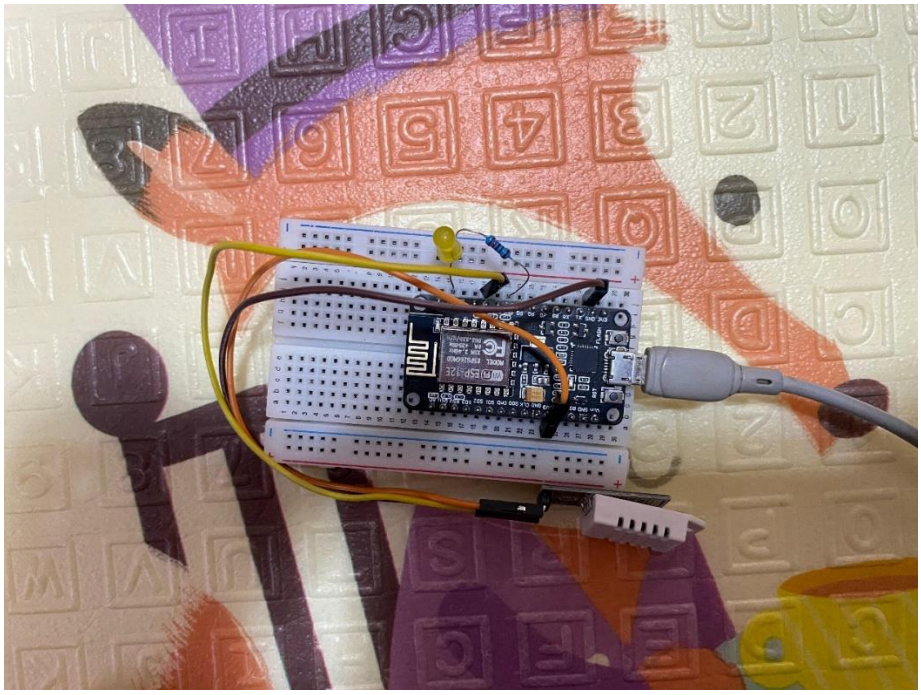


6. Hasil pengujian lanjutan pada rentang histeresis

Pengujian lanjutan dilakukan untuk memastikan bahwa LED tetap mempertahankan status sebelumnya saat suhu berada di antara dua batas histeresis. Dari data yang muncul, saya dapat mengamati perubahan suhu tanpa membuat status LED terus berganti setiap kali nilai suhu sedikit berubah. Hasil ini memperlihatkan perbedaan antara kendali dengan satu batas suhu dan kendali histeresis yang menggunakan dua batas.



7. Rangkaian Fisik



Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):
Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

Jawab

jika baris volume ./nodered:/data dihapus dari docker-compose.yml, konfigurasi dan flow Node-RED yang dibuat bisa hilang setelah container dihapus dan dibuat kembali. Hal ini terjadi karena data baru disimpan di dalam sistem file container, bukan lagi pada folder nodered di laptop yang tetap tersedia setelah container dihapus.

Hal serupa dapat terjadi pada data Mosquitto jika pemetaan ./mosquitto/data:/mosquitto/data dihapus dan data hanya tersimpan di dalam container. Karena itu, volume diperlukan agar data penting tetap tersimpan meskipun container dihentikan atau dibuat ulang.

Perbedaan Bind Mount dan Named Volume menurut saya adalah:

- Bind Mount menghubungkan folder atau file tertentu pada laptop dengan lokasi di dalam container. Contohnya ./nodered:/data, sehingga data Node-RED tersimpan pada folder nodered di laptop.
- Named Volume merupakan penyimpanan yang dikelola oleh Docker. Data disimpan terpisah dari siklus hidup container, tetapi lokasinya dikelola oleh Docker, bukan ditentukan langsung sebagai folder proyek seperti pada Bind Mount.

Jadi, keduanya bisa digunakan untuk mempertahankan data ketika container dibuat ulang. Namun, data tetap perlu dikelola dengan benar agar tidak hilang akibat penghapusan volume atau perubahan konfigurasi yang salah.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

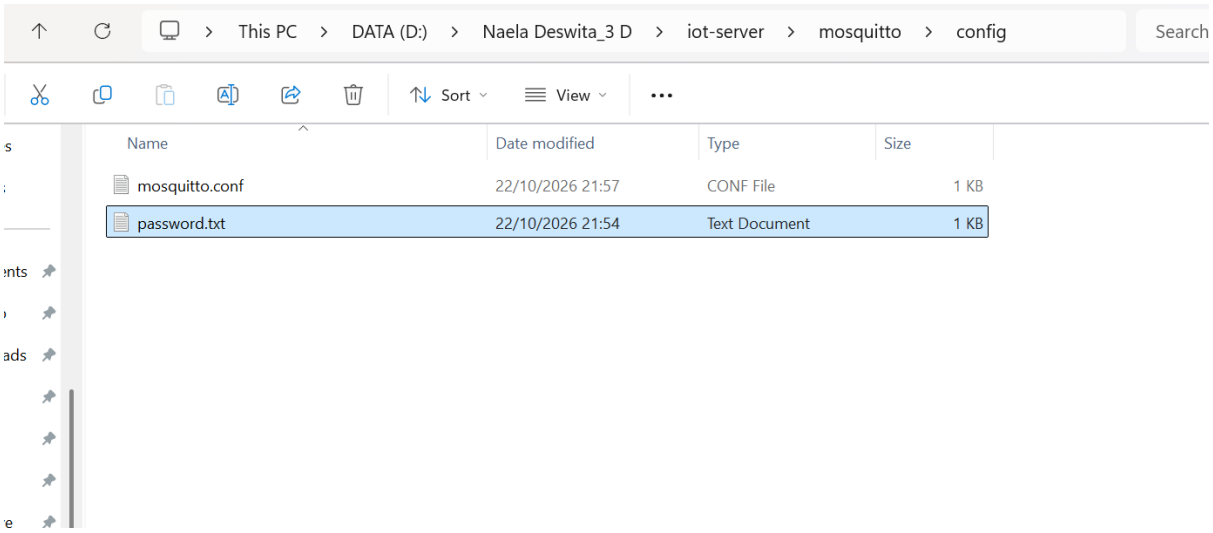
1. Kode Program Arduino Setelah Ditambahkan Autentikasi

Saya menggunakan kembali program Arduino dari Praktikum 2, kemudian memperbarui bagian koneksi MQTT agar menyertakan username dan password. Perubahan ini dilakukan karena broker Mosquitto sudah dikonfigurasi untuk menolak koneksi anonim. Dengan adanya autentikasi, NodeMCU harus menggunakan kredensial yang sesuai agar dapat terhubung kembali ke broker dan mengirimkan data sensor.



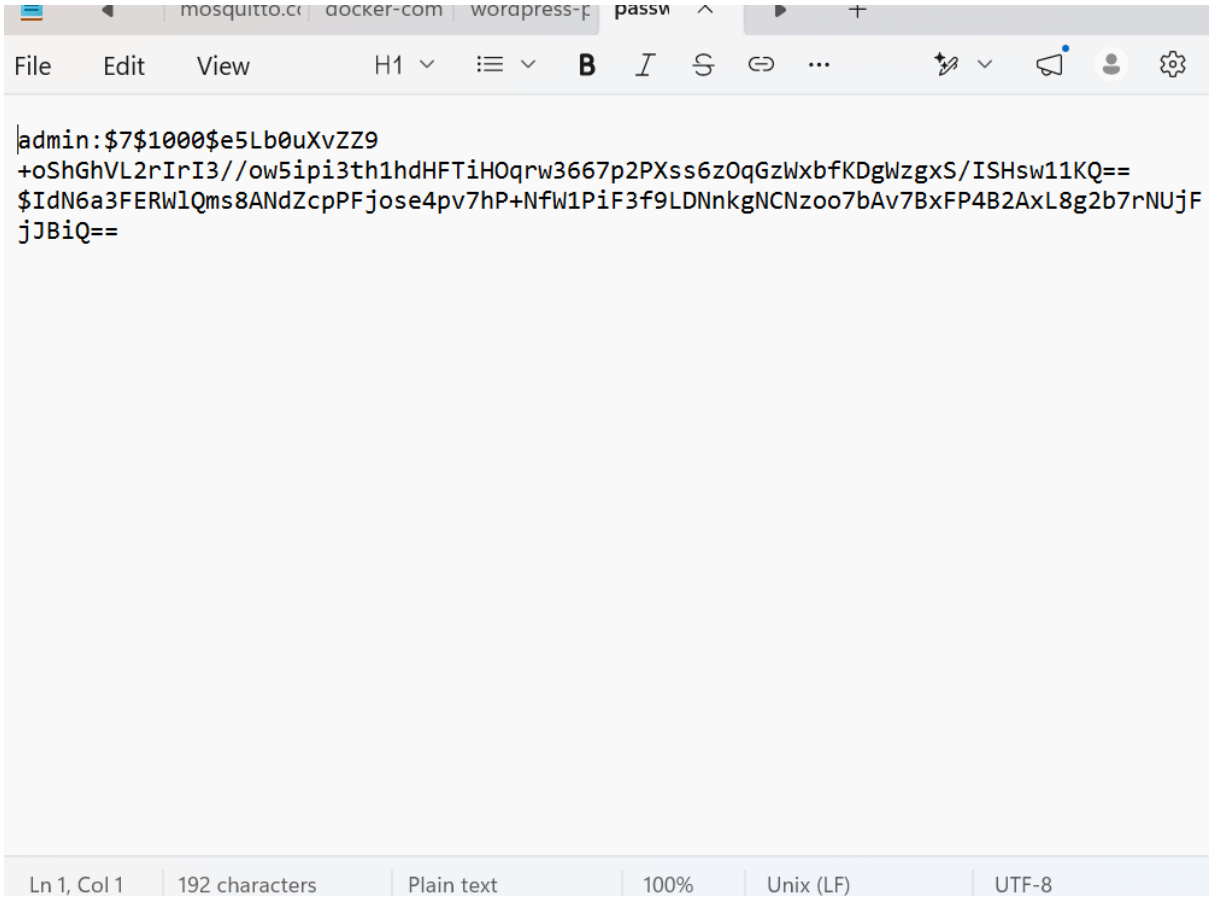
2. Pembuatan File password.txt

Saya membuat file password.txt untuk menyimpan informasi autentikasi yang akan digunakan oleh broker Mosquitto. File ini dibuat menggunakan utilitas mosquitto_passwd dengan akun pengguna yang telah ditentukan. Keberadaan file tersebut diperlukan agar broker dapat memeriksa kredensial ketika NodeMCU atau Node-RED mencoba terhubung.



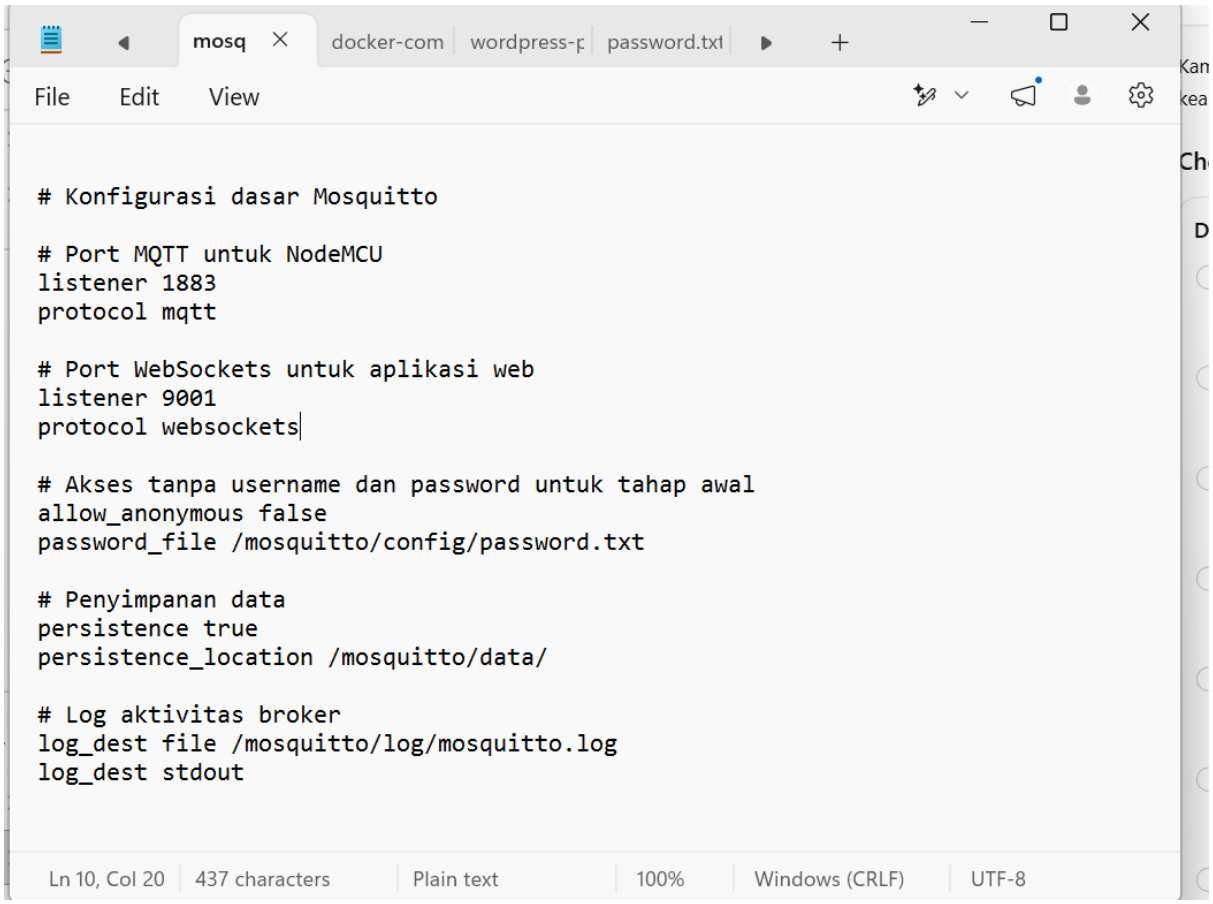
3. Isi File password.txt dalam Bentuk Hash

File password.txt berisi informasi kata sandi dalam bentuk hash, bukan kata sandi asli yang ditampilkan sebagai teks biasa. Informasi ini digunakan oleh Mosquitto untuk memeriksa kecocokan kata sandi ketika klien melakukan autentikasi. Dengan demikian, kata sandi tidak disimpan secara langsung dalam bentuk teks yang mudah dibaca.



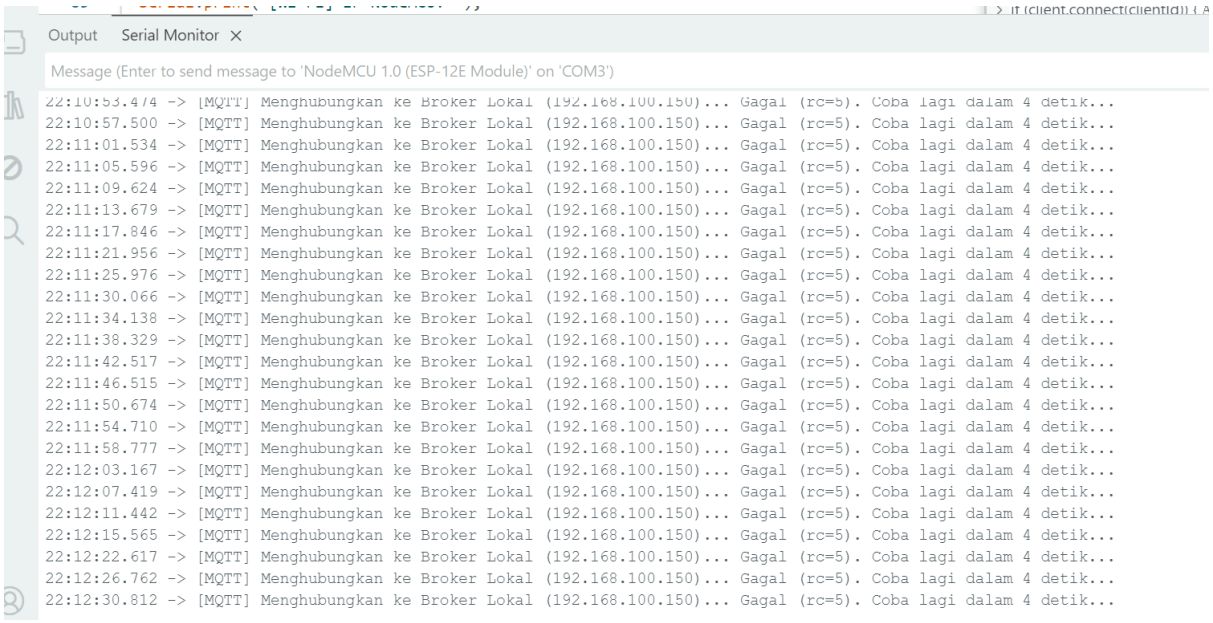
4. Konfigurasi mosquitto.conf Setelah Autentikasi Diaktifkan

Saya memperbarui file mosquitto.conf dengan menetapkan allow_anonymous false dan menambahkan konfigurasi password_file /mosquitto/config/password.txt. Pengaturan ini membuat broker tidak lagi menerima koneksi anonim. Klien yang ingin terhubung harus menggunakan username dan password yang sesuai dengan file kredensial yang telah dibuat.

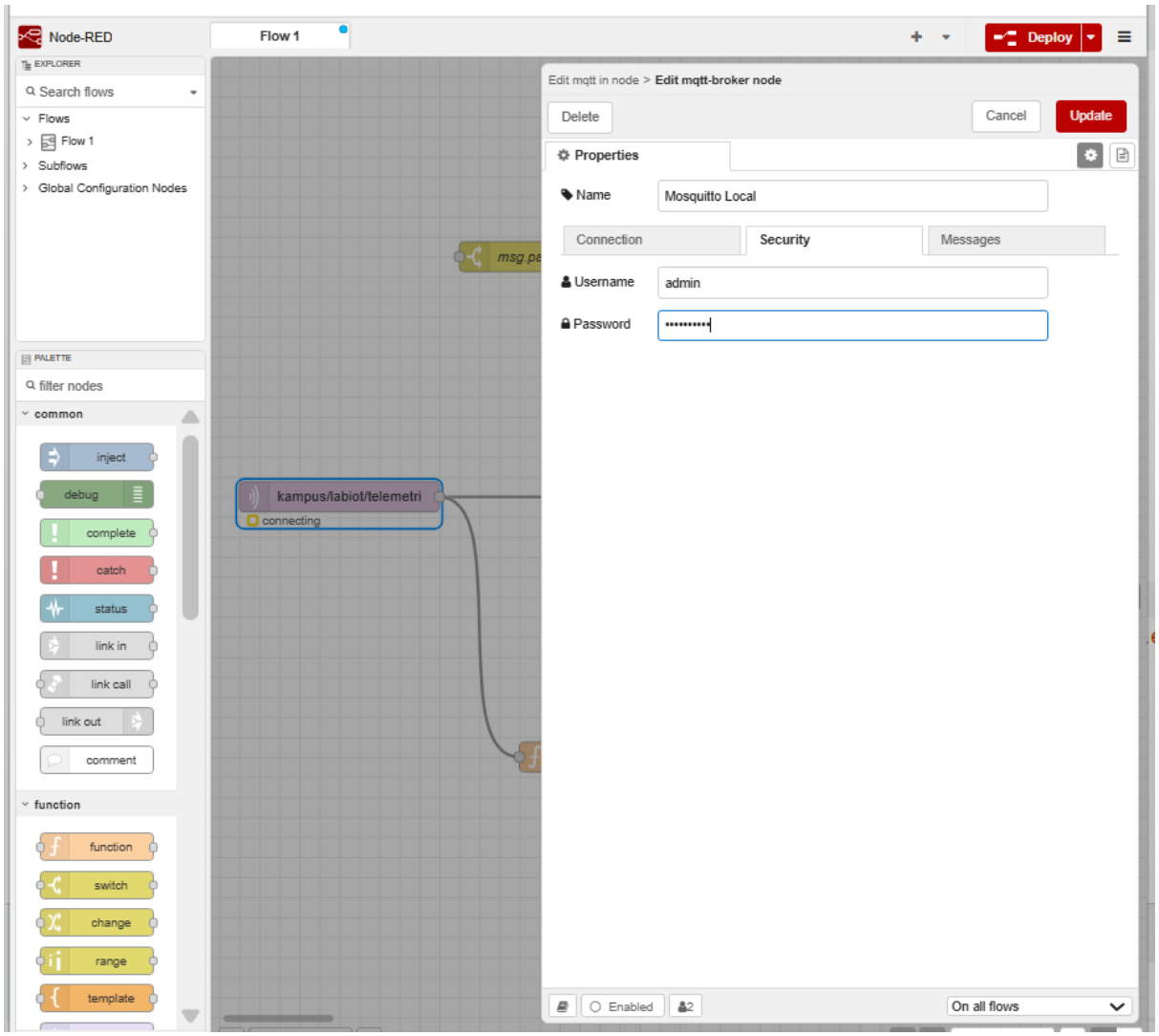


5. Bukti Koneksi Tanpa Autentikasi Ditolak

Setelah aturan keamanan diterapkan, saya memeriksa log Mosquitto untuk melihat respons broker terhadap klien yang belum menggunakan kredensial yang benar. Log menunjukkan pesan not authorised, sedangkan Serial Monitor sempat menampilkan kode penolakan rc=5. Hasil tersebut menunjukkan bahwa broker menolak koneksi yang tidak memiliki otorisasi sesuai konfigurasi.

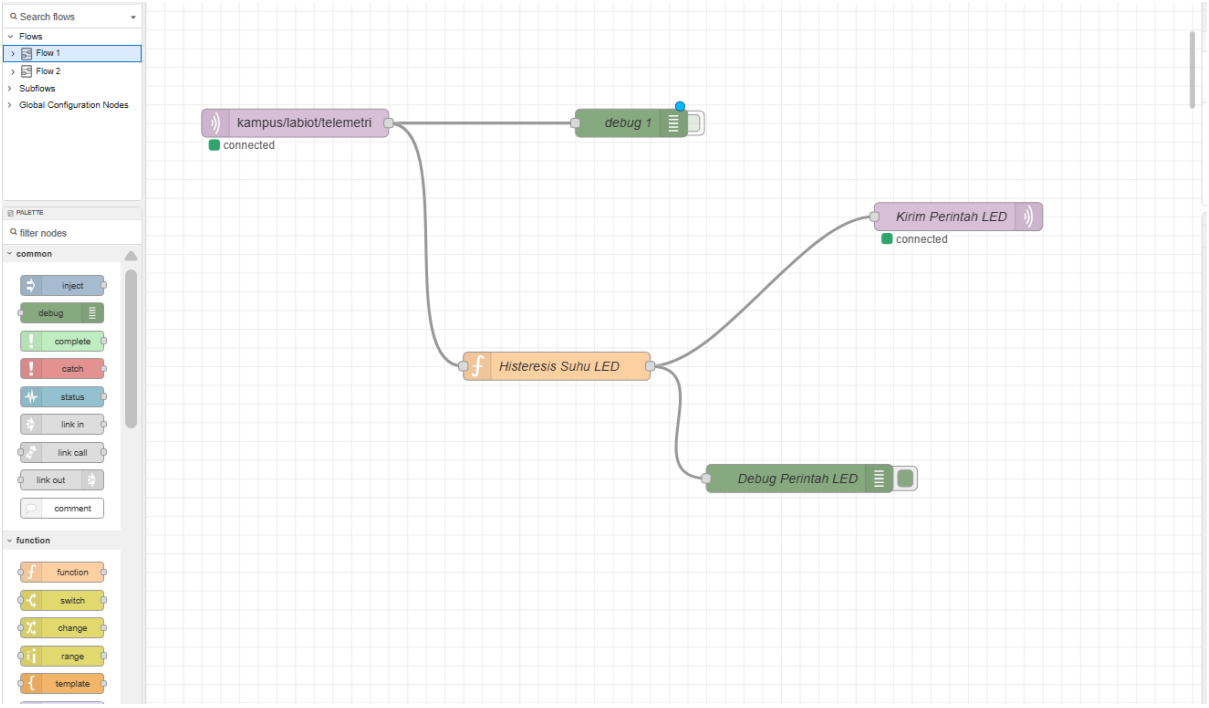


6. Konfigurasi autentikasi Node-RED



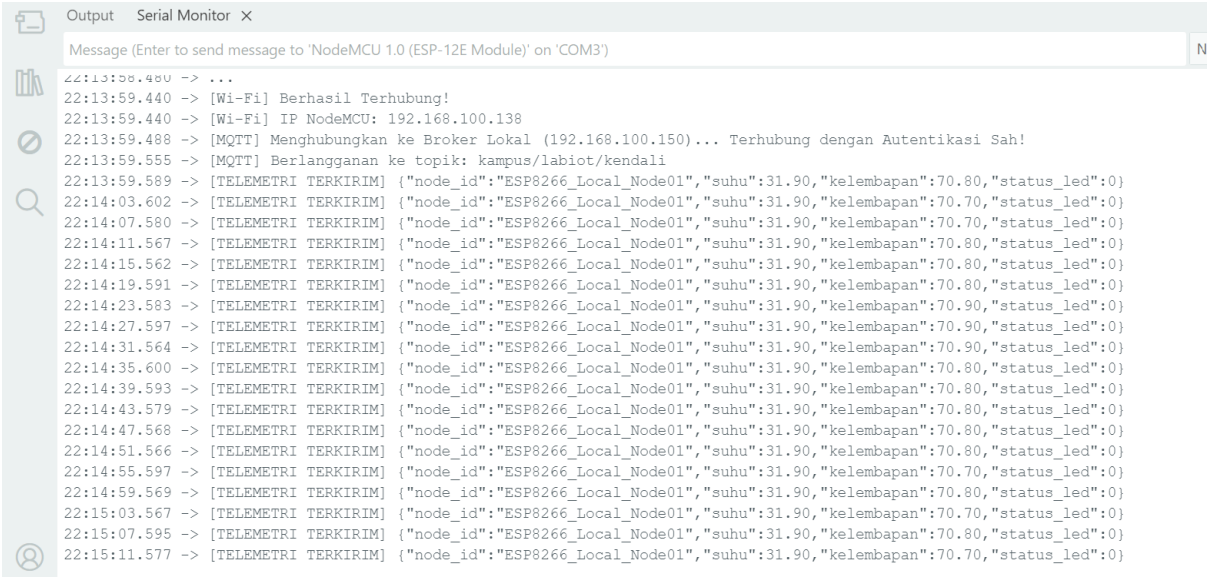
7. Flow Node-RED kembali terhubung

Saya memperbarui konfigurasi broker Mosquitto Local pada Node-RED dengan username dan password yang sesuai. Setelah perubahan disimpan dan tombol Deploy ditekan, node MQTT In dan MQTT Out kembali menunjukkan status connected. Hal ini menandakan bahwa Node-RED sudah dapat berkomunikasi dengan broker setelah autentikasi dikonfigurasi dengan benar.



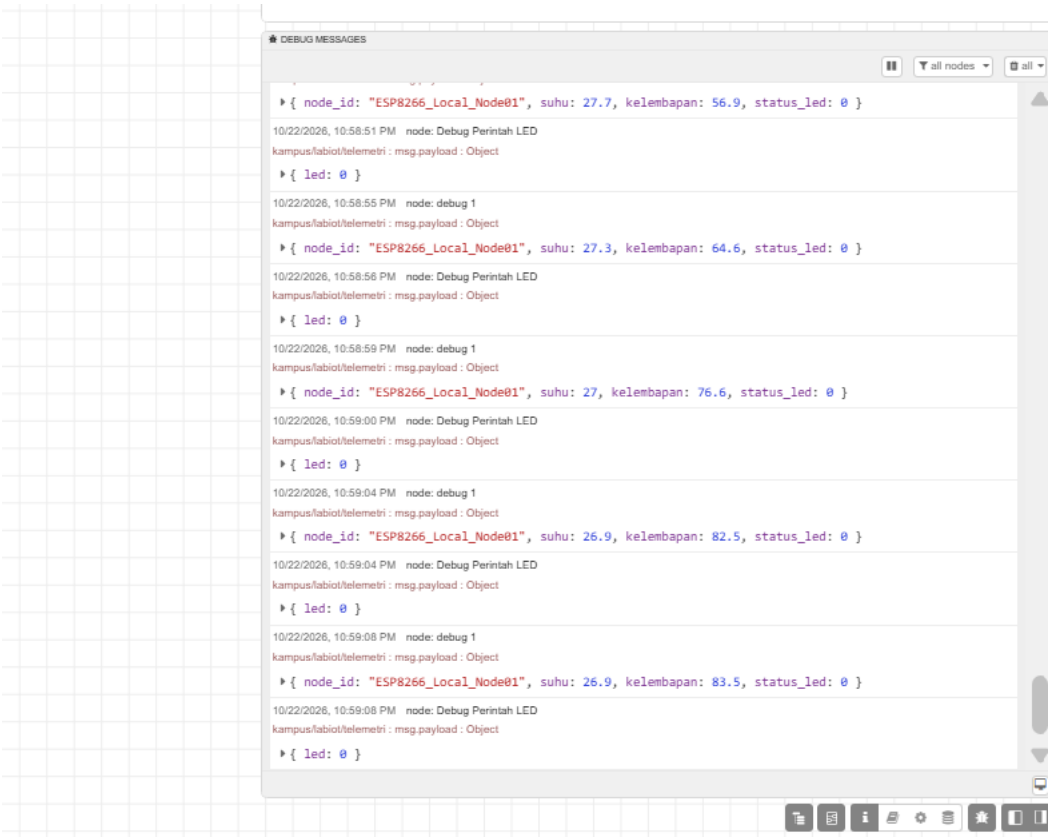
8. Serial Monitor setelah autentikasi berhasil

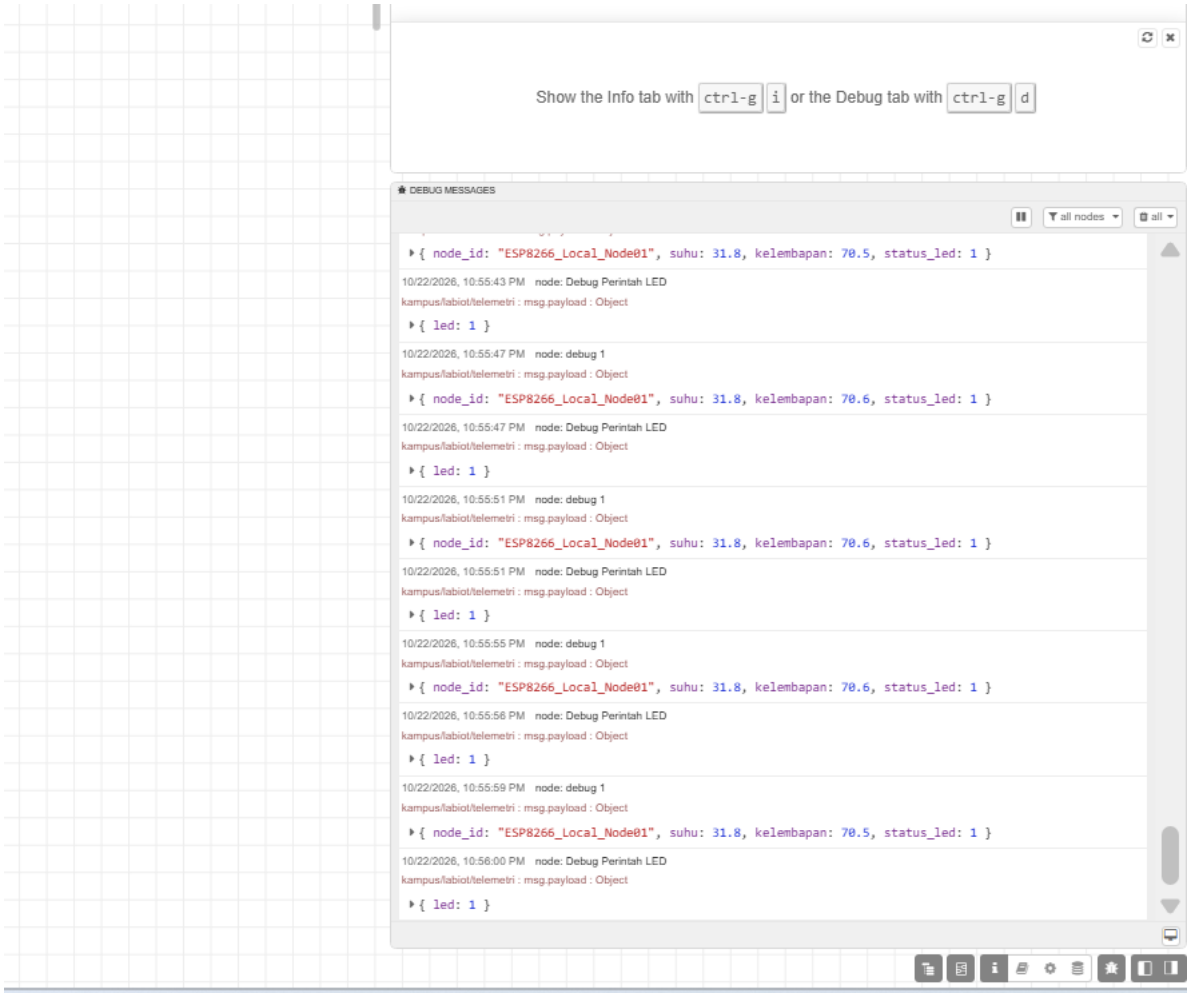
Setelah bagian koneksi MQTT pada program Arduino diperbarui, saya mengunggah ulang kode ke NodeMCU dan memeriksa hasilnya melalui Serial Monitor. Pesan Terhubung dengan Autentikasi Sah! muncul ketika perangkat berhasil terhubung menggunakan kredensial yang benar. Program kemudian berlangganan ke topik kampus/labiot/kendali dan kembali mengirimkan data telemetri secara berkala. Hasil ini menunjukkan bahwa koneksi perangkat berhasil dipulihkan setelah autentikasi diterapkan



9. Debug Messages Menampilkan Data Telemetri

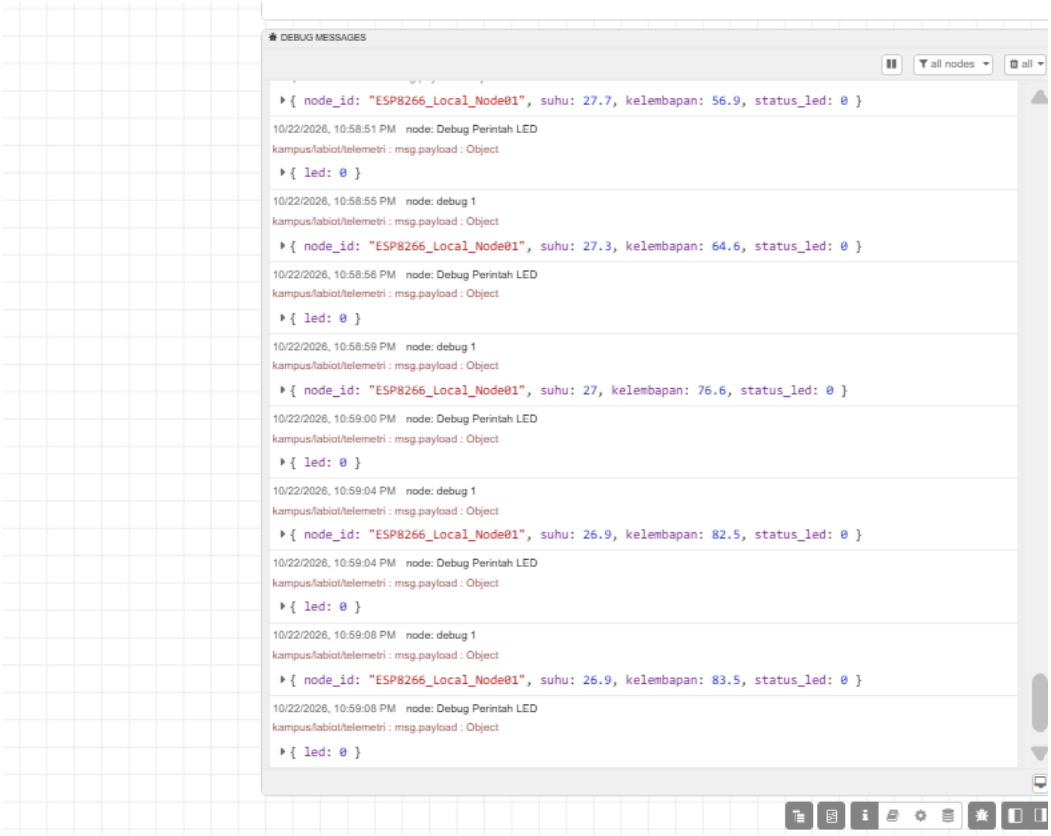
Untuk memastikan komunikasi kembali berjalan, saya memeriksa panel Debug Messages pada Node-RED. Data yang ditampilkan berisi informasi node_id, suhu, kelembapan, dan status LED. Hasil ini menunjukkan bahwa pesan telemetri dari NodeMCU kembali diterima oleh Node-RED setelah koneksi MQTT dipulihkan.

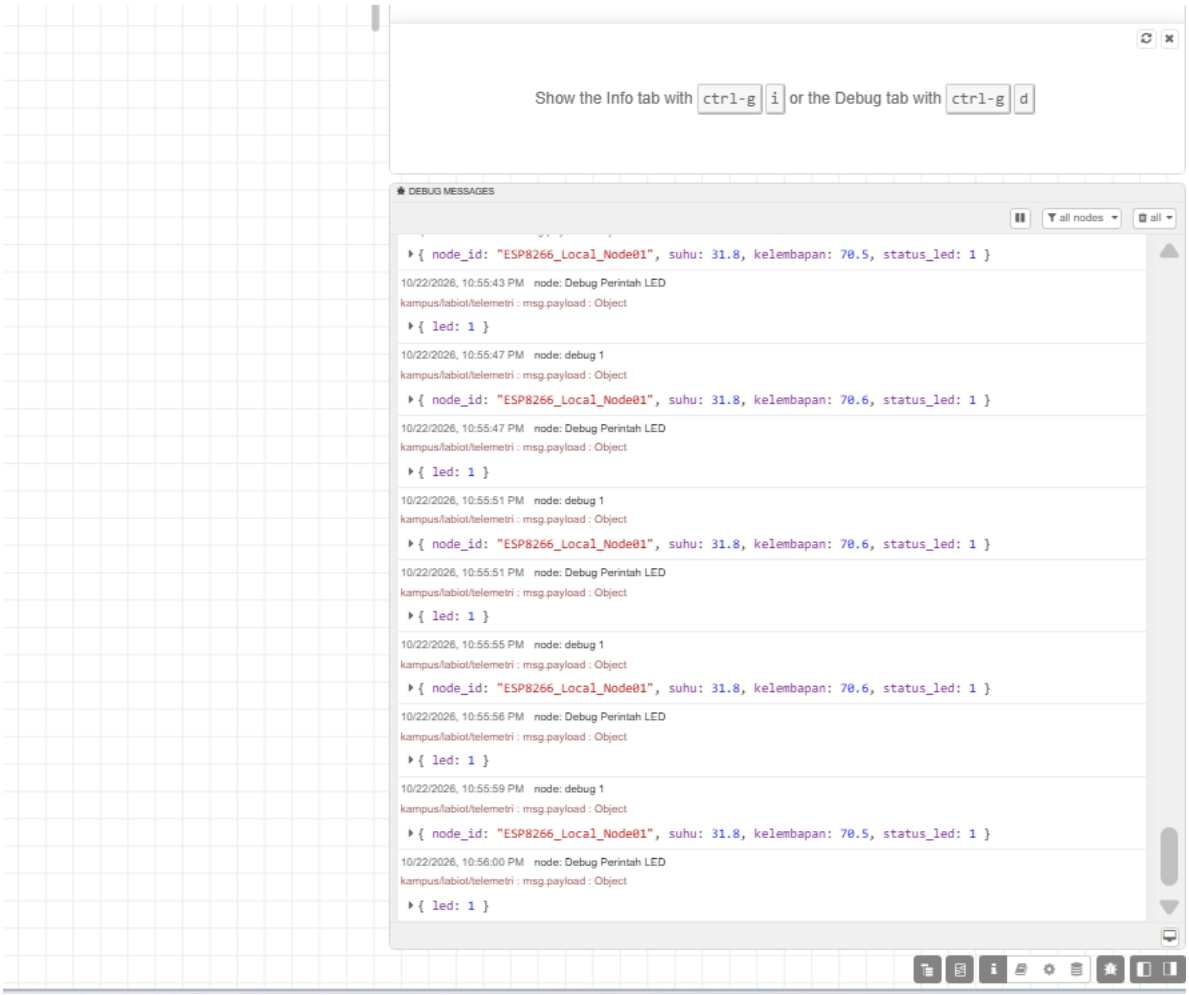




10. Debug Messages Menampilkan Perintah LED

Selain data telemetri, saya memeriksa pesan keluaran dari alur kendali LED. Pesan {"led":1} menunjukkan bahwa sistem menghasilkan perintah untuk menyalakan LED, sedangkan {"led":0} digunakan untuk memadamkannya. Pesan ini merupakan bukti bahwa aturan kendali menghasilkan instruksi yang akan diteruskan melalui topik MQTT.


DEBUG MESSAGES
all nodes
all
{ node_id: "ESP8266_Local_Node01", suhu: 27.7, kelembapan: 56.9, status_led: 0 }
10/22/2026, 10:58:51 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 0 }
10/22/2026, 10:58:55 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.3, kelembapan: 64.6, status_led: 0 }
10/22/2026, 10:58:56 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 0 }
10/22/2026, 10:58:59 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27, kelembapan: 76.6, status_led: 0 }
10/22/2026, 10:59:00 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 0 }
10/22/2026, 10:59:04 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 26.9, kelembapan: 82.5, status_led: 0 }
10/22/2026, 10:59:04 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 0 }
10/22/2026, 10:59:08 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 26.9, kelembapan: 83.5, status_led: 0 }
10/22/2026, 10:59:08 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 0 }


Show the Info tab with ctrl-g i or the Debug tab with ctrl-g d
DEBUG MESSAGES
all nodes
all
{ node_id: "ESP8266_Local_Node01", suhu: 31.8, kelembapan: 70.5, status_led: 1 }
10/22/2026, 10:55:43 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 1 }
10/22/2026, 10:55:47 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.8, kelembapan: 70.6, status_led: 1 }
10/22/2026, 10:55:47 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 1 }
10/22/2026, 10:55:51 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.8, kelembapan: 70.6, status_led: 1 }
10/22/2026, 10:55:51 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 1 }
10/22/2026, 10:55:55 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.8, kelembapan: 70.6, status_led: 1 }
10/22/2026, 10:55:56 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 1 }
10/22/2026, 10:55:59 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.8, kelembapan: 70.5, status_led: 1 }
10/22/2026, 10:56:00 PM node: Debug Perintah LED
kampus/labiot/telemetri : msg.payload : Object
{ led: 1 }

11. Hasil Serial Monitor setelah Autentikasi MQTT

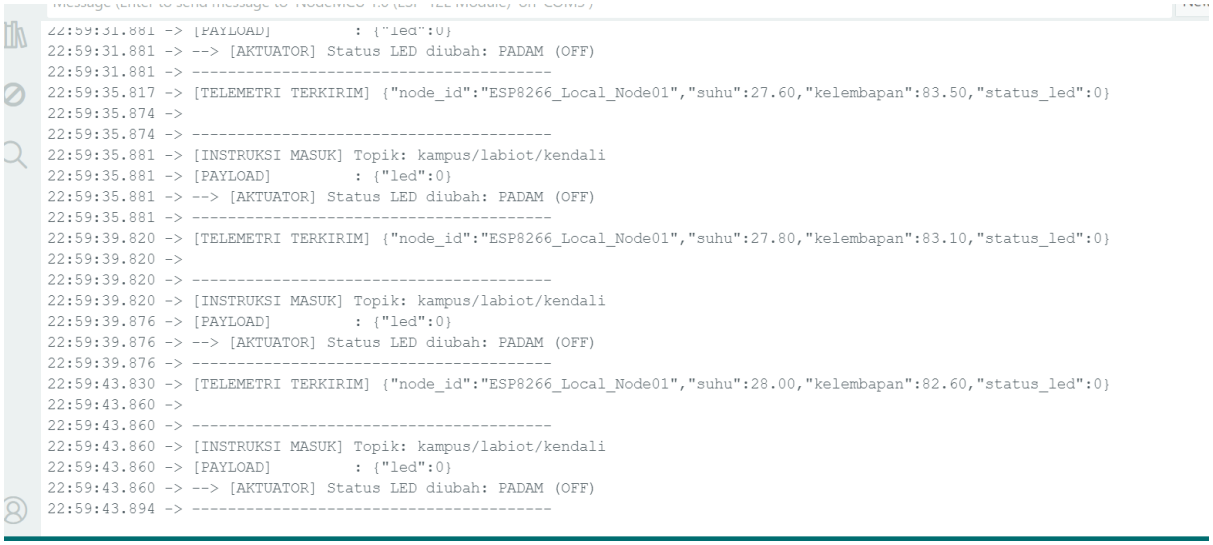
A. Pengujian saat LED ON

Saat pengujian LED menyala, Serial Monitor menampilkan pesan yang diterima NodeMCU dari topik kampus/labiot/kendali dengan payload {"led":1}. Perintah tersebut berasal dari flow Node-RED yang memproses data suhu menggunakan logika histeresis. Ketika suhu naik melebihi 31,5°C, Function Node mengirimkan perintah ON melalui broker MQTT. Setelah menerima pesan tersebut, program Arduino membaca nilai led sebesar 1, kemudian mengatur pin LED ke kondisi HIGH sehingga LED menyala. Data telemetri berikutnya juga menunjukkan status_led:1. Hasil ini membuktikan bahwa perintah kendali tetap dapat diterima dan dijalankan setelah autentikasi MQTT diterapkan.



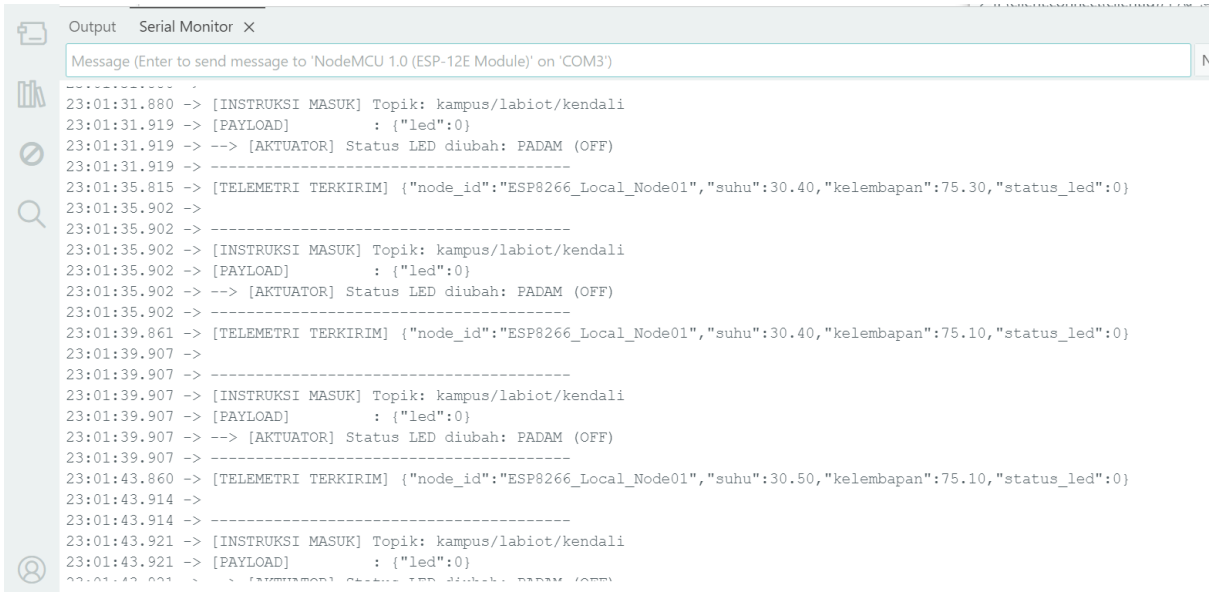
B. Pengujian saat LED OFF

Pengujian berikutnya dilakukan ketika suhu turun hingga di bawah 30°C. Pada kondisi tersebut, Function Node di Node-RED mengirimkan payload {"led":0} melalui topik kendali MQTT. NodeMCU menerima pesan tersebut, membaca nilai led sebesar 0, lalu mengatur pin LED ke kondisi LOW sehingga LED padam. Serial Monitor menampilkan keterangan Status LED diubah: PADAM (OFF), dan data telemetri berikutnya menunjukkan status_led:0. Hasil ini menunjukkan bahwa perintah OFF berhasil diproses oleh perangkat setelah koneksi menggunakan autentikasi yang sah.



C. Logika perubahan status LED

Berdasarkan kedua pengujian tersebut, sistem menggunakan suhu sebagai dasar untuk menentukan kapan LED dinyalakan atau dimatikan. LED menyala ketika suhu lebih dari 31,5°C dan padam ketika suhu turun di bawah 30°C. Jika suhu berada di antara 30°C sampai 31,5°C, LED mempertahankan status sebelumnya sehingga tidak mudah berkedip ketika suhu berubah sedikit. Setelah perintah ditentukan oleh Node-RED, pesan dikirim melalui broker Mosquitto menuju NodeMCU, lalu dijalankan oleh program Arduino. Dengan begitu, Serial Monitor membantu saya melihat hubungan antara pembacaan suhu, perintah MQTT, dan perubahan status LED.

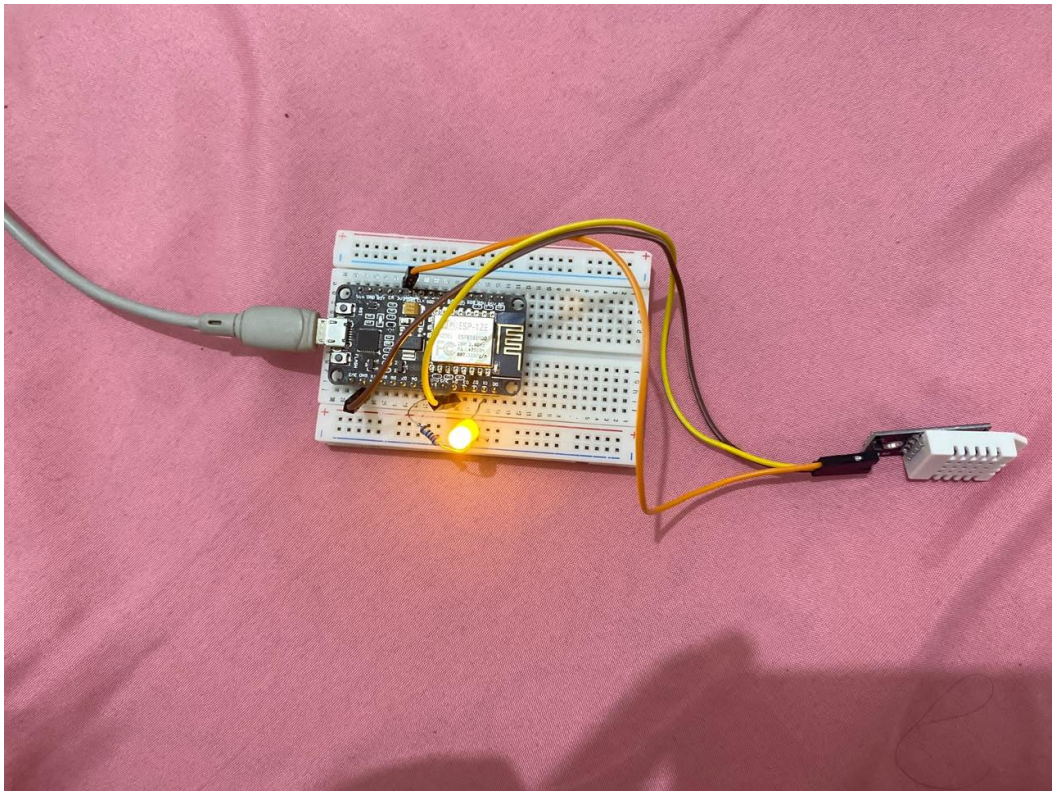


12. Rangkaian fisik LED ON dan OFF

A. Rangkaian fisik ketika LED ON

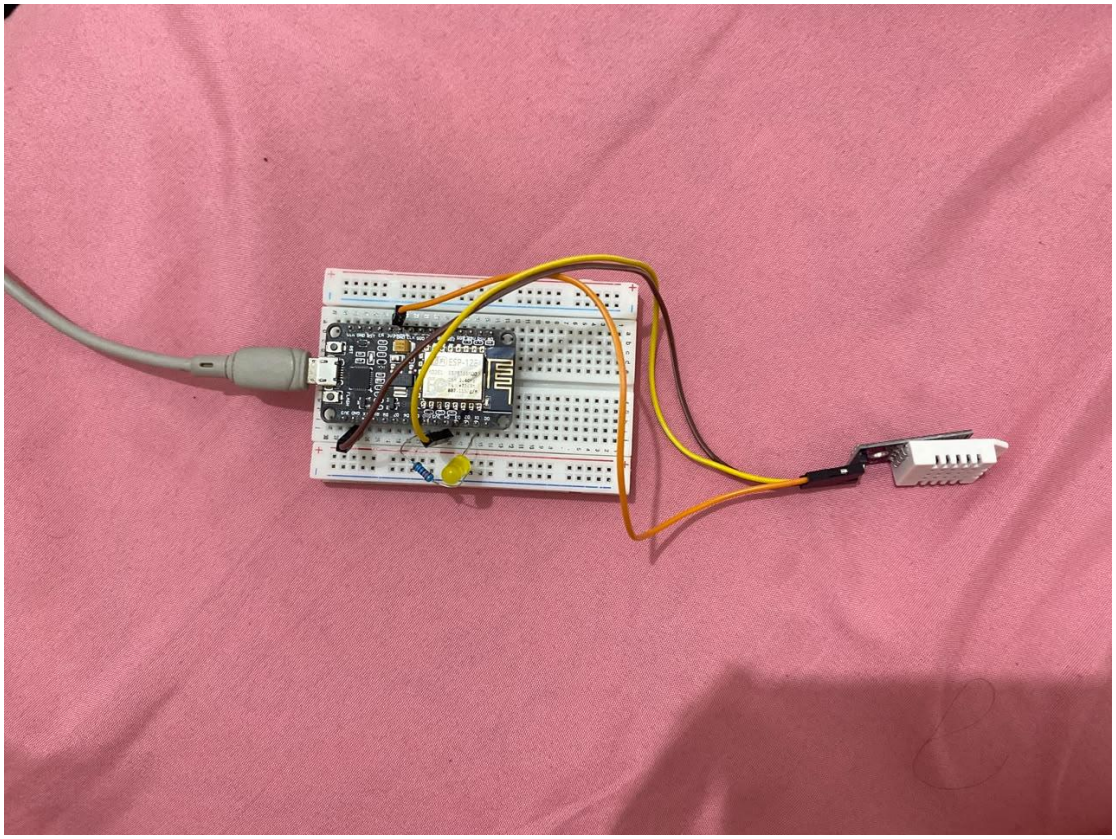
Hasil pengujian pada perangkat terlihat ketika LED indikator menyala setelah NodeMCU menerima perintah {"led":1} dari broker MQTT. Kondisi ini sesuai dengan pesan yang ditampilkan pada Serial Monitor, yaitu status LED berubah menjadi MENYALA (ON). Foto

rangkaian tersebut menjadi bukti bahwa perintah dari Node-RED berhasil diterapkan pada LED secara langsung.



B. Rangkaian fisik ketika LED OFF

Ketika NodeMCU menerima perintah {"led":0}, LED indikator berubah menjadi padam, sedangkan NodeMCU dan sensor tetap terhubung ke sumber daya. Kondisi ini sesuai dengan hasil Serial Monitor yang menunjukkan status PADAM (OFF). Hasil pengujian tersebut memperlihatkan bahwa sistem dapat mengendalikan LED dalam dua keadaan setelah koneksi MQTT diamankan menggunakan autentikasi.



Kesimpulan

Setelah menyelesaikan praktikum Modul 7, saya jadi lebih memahami bagaimana perangkat IoT bisa terhubung dengan server lokal dan saling bertukar data. Saya berhasil menjalankan Mosquitto dan Node-RED menggunakan Docker, lalu menghubungkan NodeMCU ESP8266 dengan broker MQTT lokal. Data suhu dan kelembapan dari sensor DHT bisa dikirim dan ditampilkan di Node-RED, sedangkan LED dapat dikendalikan berdasarkan aturan suhu yang dibuat.

Saya juga mencoba menerapkan logika histeresis supaya LED tidak sering berubah kondisi ketika suhu naik turun di sekitar batas tertentu. Selain itu, pada Tugas 7.7 saya mengaktifkan autentikasi MQTT dengan username dan password. Setelah koneksi tanpa kredensial ditolak, saya memperbarui pengaturan Arduino dan Node-RED sampai keduanya bisa terhubung kembali dan otomasi LED berjalan lagi.

Dari semua praktikum ini, saya jadi tahu bahwa membuat sistem IoT bukan hanya soal merangkai komponen dan menulis kode, tetapi juga memastikan komunikasi antarperangkat berjalan dengan benar dan aman. Saya juga belajar bahwa konfigurasi server, koneksi MQTT, dan aturan kendali harus saling sesuai supaya sistem bisa bekerja dengan baik.